

Unidad Didáctica

SCRUM, Unidad II

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	4
Principios.....	4
Organización.....	12
Justificación del negocio	23
Calidad	32
Cambio.....	43
Riesgo.....	51
Inicio.....	63
Planificación y estimación	70
Implementación	81
Revisión y retrospectiva.....	87
Lanzamiento	97
Bibliografía y Webgrafía	111
Glosario	112

Introducción

Scrum es un marco de trabajo ágil ampliamente utilizado en el desarrollo de software y la gestión de proyectos de ingeniería de sistemas. Surgió como una respuesta a la necesidad de adaptación rápida a cambios en los requisitos, la entrega incremental de valor y la mejora continua de los procesos (Schwaber & Sutherland, 2020). En el contexto de la Ingeniería en Sistemas de Información, Scrum permite gestionar proyectos complejos mediante iteraciones cortas llamadas sprints, fomentando la colaboración, la transparencia y la adaptabilidad.

Esta unidad didáctica tiene como objetivo profundizar en los principios, roles, artefactos y eventos de Scrum, así como en su aplicación práctica en proyectos de desarrollo de software. Se incluyen ejemplos concretos, ejercicios y autoevaluaciones para facilitar el aprendizaje y la reflexión crítica.

Principios

Scrum se articula sobre un **modelo empírico de control**: las decisiones se basan en la observación directa del trabajo realizado y los resultados obtenidos (empirismo). Los tres pilares que sostienen este enfoque son:

- **Transparencia**: la información crítica del proceso y del producto (objetivos, estado del backlog, impedimentos, criterios de aceptación) debe ser accesible y comprensible para todos los interesados que toman decisiones. Sin transparencia, la inspección pierde eficacia.
- **Inspección**: revisiones frecuentes y formales (eventos, artefactos, métricas) permiten detectar desviaciones, errores o supuestos inválidos. La inspección debe ser realizada por personas que entiendan el contexto y con frecuencia suficiente para intervenir antes de que los problemas sean demasiado costosos.
- **Adaptación**: ante hallazgos significativos en la inspección, el equipo y los stakeholders deben ajustar los planes, prioridades o prácticas para corregir el rumbo. La adaptación efectiva exige autoridad, información y un compromiso explícito con la mejora.

Desde la ingeniería de software, el empirismo reduce incertidumbre porque transforma supuestos en datos observables —por ejemplo, transformar una hipótesis sobre rendimiento de una funcionalidad en una métrica observada tras un incremento— lo cual está asociado a reducciones de defectos detectados tardíamente y a mejora en alineación con el cliente cuando se apoya con prácticas técnicas básicas (control de versiones, pruebas automatizadas, CI).

Descripción detallada de cada pilar

1) Transparencia — ¿qué debemos garantizar y cómo medirlo?

Qué incluir como mínimo:

- Estado visible y actualizado del Product Backlog (prioridades, estimaciones, criterios de aceptación).
- Sprint Backlog con tareas y responsables (o dueño de la tarea) accesible.
- Definition of Done explícita y pública.
- Registro claro de impedimentos y decisiones (log de impedimentos/action items).

Indicadores de transparencia (ejemplos medibles):

- % de historias en backlog con criterios de aceptación claros (objetivo: $\geq 90\%$).
- Tiempo medio de actualización del Sprint Board tras cambios (objetivo: < 24 h).
- Proporción de items del backlog con evidencia mínima (mockups, aceptación, documentación): objetivo $\geq 80\%$.

Riesgos si falla la transparencia: inspecciones inválidas (datos erróneos), decisiones mal orientadas, duplicación de trabajo.

Ejemplo: En digitalización de expediente estudiantil para la Universidad Politécnica, la transparencia exige que el Product Owner tenga visibilidad de todas las solicitudes de stakeholders (Secretaría Académica, Dirección, usuarios) en un backlog organizado; si una facultad solicita un campo extra para carreras técnicas, la falta de transparencia puede llevar a que se implemente sin consensuar la estructura de datos, provocando retrabajo.

2) Inspección — frecuencia, foco y técnica

Qué inspeccionar:

- El Increment (funcionalidad entregada): ¿cumple criterios de aceptación?, ¿está desplegable?
- El proceso (cómo se trabaja en el sprint): ¿se respetaron las daily?, ¿hubo blockers no resueltos?
- Las métricas (velocity, lead time, defect density): ¿muestran tendencias preocupantes?

Frecuencia recomendada:

- Daily: inspección operativa breve (máx. 15 min).
- Sprint Review: inspección del Increment por stakeholders.
- Retrospective: inspección del proceso y prácticas.

Técnicas efectivas:

- Revisiones de aceptación con checklist.
- Demos con usuarios reales o representantes (mínimo 2 stakeholders externos por sprint).
- Revisión de logs y métricas automatizadas (builds, tests, deploys).

Ejemplo: En una app de facturación para una pyme exportadora de café, la inspección debe incluir la comprobación de que la factura electrónica cumple con requisitos del DGI; una demo con el contador de la empresa permite detectar incompatibilidades regulatorias en tiempo para adaptarlas.

3) Adaptación — tomar acciones con evidencia y prioridad

Buenas prácticas para adaptar:

- Registrar la causa raíz del hallazgo en la retrospectiva.
- Priorizar acciones que reduzcan el riesgo o incrementen valor (coste-beneficio).
- Implementar la acción en el siguiente Sprint Planning como backlog item con dueño y definición de éxito.

Tipos de adaptaciones:

- Cambios en prioridades (re-priorización del backlog).
- Ajustes procesales (modificar duración de daily, cambiar definición de Done).
- Refuerzo técnico (dedicar partes del sprint a deuda técnica o a CI).

Ejemplo: Si en una cooperativa de Matagalpa se detecta que los tablets usados en campo no pueden sincronizar fotos por baja cobertura, la adaptación puede ser: implementar sincronización asíncrona y compresión de imágenes y priorizar esa tarea en el siguiente sprint.

Actividad práctica (para aula o taller) — “Diagnóstico empírico en 1 sprint simulado”

Duración: 90–120 minutos (mini-sprint) o simulado en 1 día.

Objetivo: aplicar transparencia, inspección y adaptación en un mini-proyecto local (p. ej., módulo de registro de cosechas para una cooperativa).

Pasos:

1. *Preparación (15 min)*: se entrega un backlog mínimo (3–5 user stories) y roles asignados (PO, SM, Dev Team de 4–6 personas).
2. *Sprint Planning corto (15 min)*: seleccionar 1–2 historias para el mini-sprint; acordar Definition of Done reducida.
3. *Ejecución & Daily (30–40 min)*: simulación con 2–3 ciclos de trabajo (cada ciclo 10–15 min) y daily de 5–7 min donde se registran impedimentos.
4. *Sprint Review (10–15 min)*: demo de lo entregado ante un stakeholder (puede ser docente u otro grupo).
5. *Retrospective (10–15 min)*: usar la rúbrica para diagnosticar la aplicación de principios y proponer 3 acciones de adaptación.
6. *Entrega*: registro breve (máx. 1 página) con evidencias y acciones priorizadas.

Entregable esperado: rúbrica completada + 3 acciones priorizadas (con dueño y criterio de éxito).

Rúbrica: evaluación de la aplicación de los principios (para la actividad)

Puntuación sobre 12 puntos (dividida por pilar).

Transparencia (0–4 pts)

- 0–1: Backlog y board ausentes o incomprensibles.
- 2: Backlog presente pero sin criterios de aceptación claros para la mayoría.
- 3: Backlog y Sprint Board claros; Definition of Done parcial.
- 4: Backlog completo con criterios, board actualizado y Definition of Done explícita y respetada.

Inspección (0–4 pts)

- 0–1: No hubo inspecciones regulares ni evidencia de revisión.
- 2: Inspecciones superficiales (dailies formales pero sin contenido útil).
- 3: Revisión del Increment con stakeholders, métricas básicas revisadas.
- 4: Inspecciones con evidencia, métricas validadas y feedback documentado.

Adaptación (0–4 pts)

- 0–1: No se propusieron adaptaciones o las propuestas son vagas.
- 2: Se propusieron adaptaciones pero sin prioridad ni dueño.
- 3: Acciones con prioridad y dueño definidas; plan de seguimiento.
- 4: Acciones priorizadas, con dueño y criterios de éxito; una o más ya implementadas o planificadas con fechas.

Interpretación: $\geq 9/12$ = aplicación sólida; 6–8 = aceptable con puntos a mejorar;
 <6 = necesita intervención enfocada en el próximo sprint.

Checklist/Plantilla práctica (para equipos) — “Verificación rápida de los principios”

(Pegar en el tablero físico o digital)

Transparencia

- Product Backlog visible y actualizado.
- Cada historia tiene criterio de aceptación mínimo.
- Definition of Done visible y entendida por el equipo.
- Impedimentos registrados y responsables asignados.

Inspección

- Daily programada y realizada (evidencia: nota de impedimentos).
- Sprint Review con al menos 1 stakeholder externo.
- Métrica básica actualizada (velocity o burn-down).
- Artefactos del sprint disponibles para inspección (demo, logs, pruebas).

Adaptación

- Retrospective realizada y acciones registradas.
- 1–3 acciones priorizadas para el siguiente sprint con dueño.
- Ajustes procesales documentados (si aplicaron).
- Si hubo impedimentos recurrentes, se asignó acción raíz (root cause).

Consideraciones prácticas y adaptación al contexto nicaragüense

1. **Baja conectividad:** use tableros físicos + fotos del tablero para sincronizar cuando la conexión sea intermitente; registre evidencias en hojas de campo digitalizadas al llegar a conexión.
2. **Stakeholders dispersos:** en entornos con stakeholders gubernamentales o institucionales, planifique Sprint Reviews presenciales semanales o haga demos grabadas para asegurar visibilidad.
3. **Limitaciones de recursos:** priorizar en el backlog las funcionalidades que entregan mayor valor mínimo viable; usar sprints más cortos para validar hipótesis de negocio con la menor inversión posible.
4. **Cumplimiento regulatorio:** en proyectos que interactúan con regulaciones (DGI, MINSA, etc.), incorpore checkpoints de inspección específicos que incluyan revisión por el representante legal o contador.

Organización

La **organización** en el contexto de Scrum se refiere a la estructura de personas, roles, responsabilidades y mecanismos de interacción necesarios para que el marco produzca valor de manera sostenida. Scrum define explícitamente **tres roles empíricos** con límites y responsabilidades claros: Product Owner (PO), Scrum Master (SM) y Development Team (DT). Esta simplificación deliberada busca evitar ambigüedades de responsabilidad y favorecer la inspección y adaptación rápida.

Principio clave: separación de *responsabilidad por valor* (PO) y *responsabilidad por proceso* (SM) vs *responsabilidad por entrega* (DT). Esta separación reduce fricciones y clarifica decisiones en cada sprint.

Roles y responsabilidades

Product Owner (PO)

Responsabilidad esencial: maximizar el valor del producto resultante del trabajo del equipo.

Funciones concretas:

- Mantener y ordenar el Product Backlog (priorización basada en valor y riesgos).
- Definir criterios de aceptación claros para cada user story.
- Ser el punto único de decisión respecto al contenido y prioridad del Product Backlog (aunque puede delegar tareas, la responsabilidad es suya).
- Representar a stakeholders clave (clientes, usuarios, reguladores) y negociar trade-offs entre alcance, tiempo y coste.

Capacidades recomendadas: conocimiento del dominio, visión de producto, habilidades de negociación y toma de decisiones. En contextos nicaragüenses, suele requerir habilidad para gestionar relaciones con autoridades locales o representantes de cooperativas/empresas.

Límites: no debe dirigir técnicamente al equipo ni micro-gestionar tareas; estas decisiones corresponden al equipo de desarrollo.

Scrum Master (SM)

Responsabilidad esencial: asegurar que Scrum se entienda y se aplique; facilitar la eliminación de impedimentos y proteger al equipo.

Funciones concretas:

- Facilitar eventos Scrum y ayudar al equipo a aprender prácticas ágiles efectivas.
- Trabajar con la organización para remover impedimentos y barreras organizacionales (p. ej., políticas de aprobación, procesos de contratación).
- Promover autoorganización y mejora continua (coaching, facilitación de retrospectivas).
- Colaborar con PO para maximizar el valor y entender impedimentos de negocio.

Capacidades recomendadas: facilitación, coaching, conocimiento de ingeniería de software y habilidades de negociación con la gerencia. En organizaciones pequeñas puede ejercer además tareas de Agile Coach.

Límites: no es jefe del equipo; no asigna tareas a desarrolladores.

Equipo de Desarrollo (Development Team)

Responsabilidad esencial: entregar incrementos de producto potencialmente liberables que cumplan la Definition of Done.

Características clave:

- **Multifuncional:** contiene todas las habilidades necesarias (análisis, diseño, programación, pruebas, despliegue) para entregar el incremento sin depender de unidades externas.
- **Autoorganizado:** decide internamente cómo convertir items del Sprint Backlog en incrementos.
- **Tamaño recomendado:** 3–9 personas (ideal 5–7). Menor a 3 puede carecer de capacidades, mayor a 9 dificulta la comunicación.

Competencias: se recomienda perfiles T-shaped (profundidad en una disciplina, amplitud en otras), para facilitar flexibilidad en equipos pequeños típicos de PYMES o proyectos universitarios en Nicaragua.

Interacciones y límites entre roles (gobierno y decisiones)

- **PO** decide *qué* y *en qué orden* se hace.
- **DT** decide *cómo* se realiza el trabajo para cumplir con lo comprometido.
- **SM** facilita la economía del proceso y actúa como puente con la organización.

Reglas prácticas de interacción (sugeridas):

1. El PO está disponible al menos 3 veces por semana para aclaraciones durante el sprint (o designar representante).
2. El SM tiene autoridad informal para iniciar mejoras procesales y escalamiento formal para remover impedimentos estructurales.

3. El DT mantiene autonomía en la asignación de tareas; la microgestión por parte de gerencia debe evitarse.

Organización en contextos reales

1. PYME de software en Managua (ejemplo práctico)

Escenario: empresa de 8 desarrolladores que construye software administrativo para PYMEs.

Propuesta de organización:

- PO: dueño de producto (o gerente de producto) con contacto directo a 2–3 clientes clave.
- SM: rol compartido (1 SM para 2 equipos) si recursos limitados; preferible SM dedicado cuando equipos > 1.
- DT: dos equipos de 4 personas (backend, frontend, QA combinado) o un equipo de 6 según la distribución de habilidades.

Mecanismos: reunión de coordinación semanal entre PO y gerentes; revisión de impedimentos críticos escalada al directorio trimestralmente.

2. Proyecto en una cooperativa cafetalera (zona rural)

Escenario: equipo mixto (desarrolladores, representante técnico de la cooperativa, operador de campo).

Adaptaciones sugeridas:

- PO: representante de la cooperativa + consultor técnico (co-PO) si hay complejidad regulatoria o contable.
- DT: incorporar un “champion” local (operador con habilidades técnicas básicas) para pruebas de campo.
- Consideraciones: sprints más cortos para validar en campo, meetups presenciales trimestrales si la conectividad es pobre.

3. Implementación en una dependencia pública o universidad

Desafíos típicos: jerarquías rígidas, procesos de compra lentos, contratos fijos con entregables definidos.

Sugerencias prácticas:

- Acordar desde el inicio una *forma híbrida* contractual que permita entregas iterativas (ej.: entregables por sprint y criterios de aceptación claros).
- Crear un comité de gobernanza con representación de la gerencia para decisiones de prioridad.
- El SM debe tener mandato explícito para remover impedimentos operativos (p. ej., acceso a servidores, aprobación de gasto menor).

Escalamiento de Scrum (breve guía práctica)

Si el proyecto supera la capacidad de un equipo (por ejemplo, más de 9 desarrolladores), considerar modelos de escalado simples antes que frameworks complejos:

1. **Scrum of Scrums (SoS):** equipos independientes mantienen backlog común de producto, delegan coordinación a una reunión representativa (representante de cada equipo). Útil cuando hay ~2–5 equipos.
2. **Nexus (o equivalente ligero):** enfoque para integrar trabajo entre múltiples equipos con responsabilidades claras sobre integración y resolución de dependencias.
3. **Consejo de Arquitectura / Integración:** grupo pequeño que vela por coherencia técnica, reduce dependencia entre equipos.

Regla práctica: escalar sólo cuando sea necesario y mantener la comunicación y Definition of Done compartida.

Indicadores organizacionales de salud (qué medir)

- **Lead time medio** desde la aceptación en backlog hasta entrega.
- **Porcentaje de historias entregadas que cumplen DoD** en el primer intento.
- **Número de impedimentos escalados por sprint** y tiempo medio de resolución.
- **Velocidad y su variación** (desviación estándar) para detectar inestabilidad.
- **Satisfacción del stakeholder** (survey post-review, escala 1–5).

Interpretar métricas en conjunto; evitarlas como objetivo primario (lo que se mide se optimiza—y no siempre saludablemente).

Actividad práctica (para clase/taller) — “Diseña la organización Scrum para un proyecto real en Nicaragua”

Duración: 90–120 minutos.

Objetivo: aplicar criterios de organización, roles y adaptaciones en un contexto real.

Materiales: plantilla de organigrama, backlog mínimo (5–8 historias), contexto de caso (PYME, cooperativa o dependencia pública).

Pasos:

1. *Lectura del caso* (10 min).
2. *Diseño del equipo* (20–30 min): asignar roles, tamaño del equipo, competencias necesarias. Crear organigrama y reglas de interacción (p. ej., disponibilidad del PO, canales de comunicación).
3. *Plan de gobernanza* (20 min): establecer cadencia de revisiones con stakeholders, criterios de escalamiento de impedimentos, métricas a trackear.
4. *Presentación y defensa* (20–30 min): cada grupo presenta diseño (5–7 min) y recibe preguntas.
5. *Retro breve* (10 min): qué funcionó en el diseño y qué ajustarían según feedback.

Entregable: organigrama + 5 reglas operativas + plan de gobernanza (máx. 1 página).

Rúbrica para evaluar la organización propuesta

Puntuación total: 12 puntos.

1. Claridad de roles y responsabilidades (0–4)

- 0–1: roles mal definidos o contradictorios.
- 2: roles definidos pero con solapamientos significativos.
- 3: roles definidos, límites claros, pequeñas ambigüedades.
- 4: roles y límites claramente establecidos; responsabilidades y entregables alineados.

2. Ajuste al contexto (0–4)

- 0–1: diseño genérico sin considerar el contexto del caso.
- 2: consideraciones superficiales (ej.: menciona conectividad pero sin medidas).
- 3: adaptación pertinente con medidas concretas.
- 4: diseño muy bien adaptado (p. ej., propuestas concretas para baja conectividad o marco regulatorio).

3. Gobernanza y mecanismos de escalamiento (0–4)

- 0–1: sin gobernanza ni plan de escalamiento.
- 2: gobernanza presente pero incompleta.
- 3: plan de gobernanza y escalamiento claro; falta detalle en métricas.
- 4: gobernanza, escalamiento y métricas bien definidas; roles empoderados para actuar.

Interpretación: ≥ 10 = diseño listo para piloto; 7–9 = viable con ajustes; <7 = replantear con foco en roles y contexto.

Checklist / Plantilla práctica (para usar al iniciar un equipo)

(Puedes pegar esto en el board del equipo o enviarlo por correo en la fase de kickoff)

Antes del kickoff

- Identificado el Product Owner con autoridad para priorizar.
- Nombrado un Scrum Master con mandato para remover impedimentos.
- Equipo de Desarrollo formado con habilidades mínimas (dev, QA, ops) y tamaño 3–9.
- Backlog inicial con al menos 8 historias y criterios de aceptación.

Durante el kickoff

- Aprobadas reglas de interacción (PO disponible X horas/semana).
- Definida la Definition of Done mínima.
- Establecidos canales de comunicación (Slack/WhatsApp + tablero físico/virtual).
- Acordada cadencia de eventos (sprint length, dailies, reviews, retros).

Gobernanza y escalamiento

- Procedimiento documentado para escalamiento de impedimentos.
- Frecuencia de reporte a mandos (weekly/biweekly).
- Métricas a trackear y responsables.

Consideraciones prácticas y recomendaciones para implementación

1. **Formación inicial:** invertir en formación breve (workshop de 1–2 días) para PO, SM y DT; idealmente con un sprint de práctica.
2. **Mandato organizacional:** el SM necesita respaldo explícito de la gerencia para remover impedimentos que requieran cambios organizativos.
3. **Contratación/roles híbridos:** en organizaciones pequeñas, un rol puede asumir responsabilidades adicionales (p. ej., PO que además actúe como sponsor), pero documentar límites y evitar conflicto de intereses.
4. **Part-time / estudiantes:** si el equipo incluye estudiantes o personas part-time, ajustar la capacidad del sprint y priorizar mínima entrega viable para mantener ritmo.
5. **Gestión de conflictos:** establecer reglas claras para resolución de conflictos (decisión final del PO respecto al alcance, mediación por SM en conflictos de proceso).
6. **Legal y contratos:** para contratos con la administración pública, negociar cláusulas que permitan entregas por iteraciones y criterios de aceptación por sprint o release.

Medios de verificación y métricas de éxito (piloto inicial)

- Resultado del primer piloto (3 sprints): porcentaje de historias entregadas que cumplen DoD $\geq 70\%$.
- Tiempo medio de resolución de impedimentos ≤ 3 días para problemas operativos; ≤ 14 días para impedimentos estructurales (requieren escalamiento).
- Satisfacción de stakeholders (post-review): promedio $\geq 4/5$.
- Retrospectivas documentadas y con al menos 1–2 acciones concretas implementadas por sprint a partir del segundo sprint.

Justificación del negocio

La **justificación del negocio** es el argumento (técnico, económico y estratégico) que demuestra que un proyecto merece la inversión de recursos. No es sólo “hacer bonito en la presentación”; su objetivo real es:

- Probar que los beneficios esperados superan (con margen razonable) los costos y riesgos.
- Priorizar iniciativas cuando recursos son limitados.
- Alinear el proyecto con objetivos organizacionales y regulaciones locales.
- Proveer métricas para tomar decisiones durante el ciclo de vida del proyecto (empirismo aplicado al negocio).

En contextos nicaragüenses —PYMEs, cooperativas agrícolas, universidades públicas— la justificación debe incluir además impactos sociales (empleo local, capacitación), restricciones operativas (conectividad), y requerimientos regulatorios (p. ej., facturación electrónica con DGI).

Metodología práctica para elaborar la justificación

Paso 1 — Diagnóstico del problema y alcance

- Describe la situación actual: procesos manuales, errores frecuentes, tiempos de respuesta, cuellos de botella.
- Define el alcance del proyecto: entregables principales, límites, stakeholders.

Paso 2 — Identificar y cuantificar beneficios

- **Beneficios directos** (ahorro en horas de trabajo, reducción de errores, ingresos adicionales).
- **Beneficios indirectos** (mejor toma de decisiones, mejora en satisfacción del cliente, cumplimiento regulatorio que evita multas).
- Cuantificar en unidades monetarias cuando sea posible (USD o Córdobas), o bien en indicadores que luego se monetizan (horas hombre * costo por hora).

Paso 3 — Estimar costos

- **Costos de capital (one-time)**: desarrollo, licencias, hardware.
- **Costos operativos anuales**: mantenimiento, hosting, soporte, actualizaciones.
- **Costos de transición**: capacitación, migración de datos, gestión del cambio.

Paso 4 — Calcular métricas financieras simples

- **Beneficio anual bruto (B_a)** = suma de beneficios monetizados por año.
- **Costo anual neto (C_a)** = costos operativos anuales.
- **Beneficio neto anual (BN_a)** = $B_a - C_a$.
- **Período de recuperación (payback)** = $C_{\text{capital}} / BN_a$ (si $BN_a > 0$).
- **ROI simple anual (primer año)** = $(B_a - C_{\text{capital}} - C_a) / C_{\text{capital}}$.
- **ROI acumulado N años** = $(\sum_{t=1..N} BN_a - C_{\text{capital}}) / C_{\text{capital}}$.

(En el ejemplo abajo muestro los cálculos paso a paso con números concretos).

Paso 5 — Análisis de sensibilidad

- Construir al menos 2 escenarios: *Base* (estimaciones prudentes), *Optimista* (mejoras posibles), y opcional *Pesimista*.

- Evaluar cómo varía payback y ROI según cambios razonables en horas ahorradas, costo de desarrollo o mantenimiento.

Paso 6 — Evaluación cualitativa y decisión

- Razonar aspectos no monetizados: cumplimiento legal, reputación, riesgos reputacionales, impacto social.
- Recomendar: aprobar (si indicadores financieros y no financieros favorables), aprobar con condiciones (p. ej., pilotar 1 sprint), o rechazar.

Ejemplo práctico y completo — *Digitalización de inventario* para una cooperativa en Matagalpa

Contexto resumido

La cooperativa X en Matagalpa gestiona inventario de sacos de café de forma manual: dos personas registran entradas y salidas en cuadernos y hojas de cálculo; hay errores y retrasos en la información que afectan ventas y trazabilidad. Se propone desarrollar un sistema móvil+servidor que permita captura en campo, sincronización asíncrona y generación de reportes y códigos QR para identificar lotes.

Supuestos (escenario base)

- **Costos de capital (desarrollo):** USD **10 000** (one-time).
- **Costos operativos anuales (hosting, soporte, actualizaciones):** USD **1 200 / año**.
- **Personal actual dedicado a inventario:** 2 personas.
 - Salario promedio estimado por persona: USD **350 / mes** → **4 200 / año** por persona.
 - Proporción de tiempo dedicada a tareas de inventario: **50%**.
 - Coste anual atribuible a inventario = $2 * 4\,200 * 0.5 =$ **USD 4 200**.
- **Reducción estimada de carga laboral por automatización: 70%** (con proceso digitalizado).
 - Ahorro anual en mano de obra = $4\,200 * 0.70 =$ **USD 2 940**.
- **Reducción de pérdidas/errores (valor recuperado): USD 1 000 / año** (menos devoluciones, menos inventario errado).

Con estos supuestos calculamos:

Beneficio anual bruto (B_a) = ahorro en mano de obra + reducción de errores
= 2 940 + 1 000 = **USD 3 940 / año.**

Beneficio neto anual (BN_a) = B_a – costos operativos anuales
= 3 940 – 1 200 = **USD 2 740 / año.**

Período de recuperación (payback) = Coste de capital / BN_a
= 10 000 / 2 740 ≈ **3,65 años.**

Cálculo 3-años (ROI simple acumulado):

- Beneficios acumulados en 3 años (brutos) = 3 940 * 3 = **11 820.**
- Costos operativos 3 años = 1 200 * 3 = **3 600.**
- Beneficio neto acumulado 3 años = 11 820 – 3 600 = **8 220.**
- ROI 3 años = (Beneficio neto acumulado – Coste capital) / Coste capital =
(8 220 – 10 000) / 10 000 = **-0,178 = -17,8%.**

Interpretación: bajo estos parámetros, **el proyecto se recupera en ~3,7 años** y el ROI acumulado a 3 años sería negativo (-17,8%) porque la inversión inicial todavía no se ha cubierto completamente por beneficios netos en ese horizonte — aunque la operación es rentable a largo plazo (>3.65 años). Esto es típico en proyectos con inversiones de capital moderadas y beneficios anuales modestos.

Análisis de sensibilidad (escenario optimista)

Suponiendo mejoras (mejor tasa de ahorro y mayor reducción de errores):

Supuestos optimistas:

- Ahorro laboral anual = **USD 4 200** (por ejemplo por mayor eficiencia o salario mayor evitado).
- Reducción de errores = **USD 1 500 / año**.

B_a (óptimo) = 4 200 + 1 500 = **USD 5 700**.

BN_a (óptimo) = 5 700 – 1 200 = **USD 4 500**.

Payback (óptimo) = 10 000 / 4 500 $\approx$ **2,22 años**.

ROI 3 años (óptimo) = $((5\,700 \times 3 - 1\,200) - 10\,000) / 10\,000 = 0,35 =$
35%.

Interpretación: en un escenario plausible pero más favorable, el payback baja a ~2,2 años y el ROI acumulado a 3 años es +35%—un resultado claramente atractivo.

Conclusiones del ejemplo

- El proyecto **es financieramente viable** en el mediano plazo; la decisión depende del horizonte requerido por la cooperativa y de la disponibilidad de financiación.
- Si la organización exige payback < 2 años, el proyecto debería re-dimensionarse (reducir coste de desarrollo, aumentar alcance de beneficios, o buscar subvención).
- El análisis de sensibilidad muestra que pequeñas mejoras en la operacionalización (p.ej., mayor % de reducción de tiempo o inclusión de ganancias por mayor volumen de ventas) mejoran notablemente la viabilidad.

KPI y métricas recomendadas para la justificación y el seguimiento

- **Beneficio neto anual (USD/año)** — para evaluar la salud financiera recurrente.
- **Período de recuperación (años)** — para decisiones de inversión.
- **% reducción de horas hombre dedicadas a la tarea** — valida supuestos de ahorro.
- **% reducción de errores/incidencias y valor recuperado.**
- **% de historias entregadas que cumplen DoD** (calidad técnica).
- **Satisfacción del stakeholder** (post-review) escala 1–5.
- **Tasa de adopción en usuarios claves** (p. ej. % de operarios que usan la solución tras 3 meses).

Matriz de riesgos (resumen) y medidas de mitigación

1. **Riesgo: Subestimación de costos de desarrollo**
 - **Impacto:** Alto. **Mitigación:** incluir 15–25% de contingencia; contratar alcance mínimo viable (MVP) y fases.
2. **Riesgo: No se alcanzan los ahorros estimados (menor adopción)**
 - **Impacto:** Alto. **Mitigación:** plan de gestión del cambio, capacitación, involucrar “champions” locales.
3. **Riesgo: Problemas de conectividad en campo**
 - **Impacto:** Medio-Alto. **Mitigación:** desarrollo offline-first con sincronización asíncrona; pruebas en condiciones reales.
4. **Riesgo: Riesgos regulatorios o de cumplimiento**
 - **Impacto:** Medio. **Mitigación:** incluir revisión legal/contador desde requisitos (especialmente si hay facturación electrónica).
5. **Riesgo: Soporte y mantenimiento insuficiente**
 - **Impacto:** Medio. **Mitigación:** contrato de mantenimiento, transferencia de conocimiento y documentación.

Aspectos no financieros que fortalecerán la justificación (muy relevantes en Nicaragua)

- **Impacto social y comunitario:** capacitación y empleo local, fortalecimiento de capacidades en cooperativas.
- **Resiliencia operativa:** menos dependencia de personas clave (documentación y procesos).
- **Cumplimiento y trazabilidad:** mejoras en cumplimiento con regulaciones (DGI, trazabilidad de café, certificaciones).
- **Competitividad:** capacidad de responder más rápido a compradores y exportadores.

Estos factores suelen inclinar la decisión cuando los indicadores financieros por sí solos son marginales.

Plantilla práctica: esquema de justificativo (copiar y completar)

1. **Título del proyecto**
2. **Resumen ejecutivo (máx. 200–300 palabras):** qué se propone y por qué.
3. **Problema / oportunidad:** descripción cuantitativa y cualitativa.
4. **Alcance del proyecto:** entregables, límites.
5. **Stakeholders principales.**
6. **Supuestos clave** (listar).
7. **Costos estimados:**
 - Costos one-time (USD) = Desarrollo: ____ ; Hardware: ____ ; Licencias: ____ ; Total: ____.
 - Costos anuales (USD) = Hosting: ____ ; Soporte: ____ ; Total anual: ____.
8. **Beneficios estimados (USD/año):**

- Ahorro en horas hombre: ____ ; Valor reducción errores: ____ ; Otros: ____.
- Beneficio anual bruto = ____ .
- Beneficio neto anual = Beneficio anual bruto – Costos anuales = ____ .

9. **Métricas calculadas:** Payback = Costos one-time / Beneficio neto anual = ____ años. ROI 1er año = (Beneficio anual bruto – Costos one-time – Costos anuales) / Costos one-time = ____%. ROI N años = ____%.

10. **Análisis de sensibilidad:** escenario base / optimista / pesimista. (tabla con 3 columnas).

11. **Riesgos principales y mitigaciones.**

12. **Recomendación:** aprobar / pilotar / rechazar + condiciones.

13. **Plan de seguimiento:** KPIs, responsables, frecuencia de reporte.

La **justificación del negocio** es tanto una exigencia técnica como un acto de comunicación: no sólo demuestra que el proyecto “cuadra” numéricamente, sino que transmite confianza a los tomadores de decisión. En contexto nicaragüense, donde los recursos y la conectividad pueden ser limitaciones importantes, una justificación sólida que combine **números claros, análisis de sensibilidad, plan de mitigación de riesgos y beneficios sociales** suele ser la fórmula ganadora.

Calidad

La calidad de software se divide habitualmente en dos dimensiones complementarias:

- **Calidad del producto (qué se entrega):** funcionalidad correcta, fiabilidad, rendimiento, seguridad, usabilidad, mantenibilidad.
- **Calidad del proceso (cómo se entrega):** prácticas repetibles y controladas que reducen la variabilidad (TDD, CI/CD, code review, automatización de pruebas).

Desde la ingeniería de software, la evidencia empírica muestra que combinar prácticas de aseguramiento (detección precoz de defectos mediante pruebas automatizadas y revisiones) con procesos iterativos (Sprints cortos, Definition of Done) disminuye el coste de corrección de errores y mejora la predictibilidad del equipo. La aproximación recomendada es integrar la calidad *en el flujo de trabajo* (shift-left), no dejarla como una fase final (shift-right únicamente).

Componentes prácticos de un plan de calidad (Quality Plan)

1. **Definition of Done (DoD)** — lista mínima que debe cumplirse para que un incremento sea considerado “hecho”.
2. **Estrategia de pruebas** — qué tipos de pruebas, responsabilidades y criterios de aceptación.
3. **Pipelines de CI/CD y quality gates** — pasos automatizados que validan builds antes de permitir merges/despliegues.
4. **Prácticas de ingeniería** — TDD/BDD, pair programming, code review, análisis estático, gestión de deuda técnica.
5. **Métricas y KPIs** — conjunto acotado de indicadores para evaluar calidad objetiva y rendimiento del proceso.

6. **Riesgos de calidad y mitigaciones** — matriz de riesgos con acciones concretas.
7. **Governance** — roles y responsabilidades (quién valida, quién corrige, escalamiento).

A continuación se presenta cada componente con plantillas y ejemplos

1) Definition of Done (plantilla y ejemplo)

Plantilla DO D (ejemplo mínimo para sistemas de información):

- Código compilado sin errores.
- Tests unitarios automáticos ejecutados y *todos* pasan.
- Cobertura mínima de pruebas unitarias: $\geq 70\%$ (ver nota sobre interpretación).
- Tests de integración automatizados ejecutados y pasan (si aplica).
- Código revisado por al menos 1 par mediante Pull Request y aprobado.
- Análisis estático de código pasa sin nuevos bloqueadores (por ejemplo, SonarQube: no new critical issues).
- Documentación mínima actualizada (README, API contract, steps de despliegue).
- Instrucciones para despliegue y rollback documentadas.
- Build exitoso en pipeline de CI y deploy a entorno de staging.
- Acceptance tests / criterios de aceptación validados y sign-off del Product Owner.
- Seguridad básica: escaneo SAST sin vulnerabilidades críticas.
- Migraciones de datos probadas (si aplica).

Ejemplo aplicado (sistema de gestión estudiantil en UPN): además de los items anteriores, añadir: auditoría de cambios activada (registro de quién modificó notas), export/import de actas testado, verificación de integridad referencial en base de datos y pruebas de carga simulando pico de matrícula (p. ej., 500 concurrentes).

Nota sobre cobertura: fijar un % arbitrario de cobertura no garantiza calidad funcional. Use cobertura como indicador de que hay pruebas, pero combínelo con revisiones de pruebas (calidad de pruebas) y pruebas de integración/aceptación. Considerar métricas complementarias (mutation testing) para evaluar la eficacia de las pruebas.

2) Estrategia de pruebas (testing pyramid y prácticas)

Testing Pyramid (prioridad y por qué):

- **Base: Tests unitarios** — rápidos, aislados, cubren lógica. Objetivo: detectar regresiones locales.
- **Medio: Tests de integración** — prueban interacción entre componentes (BD, servicios).
- **Cima: Tests end-to-end / acceptance** — simulan escenarios reales; más lentos, costosos y frágiles; deben ser pocos y representativos.

Prácticas recomendadas:

- Adoptar **TDD** (Test-Driven Development) donde sea factible: escribir tests unitarios antes del código para guiar diseño y asegurar cobertura.
- Utilizar **BDD** (Behavior Driven Development) para historias críticas o de negocio (ej., escenarios de facturación electrónica) con herramientas tipo Cucumber/Gherkin para facilitar comunicación PO-Dev-QA.
- Automatizar pruebas en CI: los builds deben ejecutar tests unitarios e integración; sólo si resultan OK avanzar a quality gates.

- Mantener **suites rápidas** (tests unitarios) que se ejecutan en cada commit y **suites lentas** (e2e, performance) que se ejecutan en nightly o en pipelines específicos.
- Implementar **test data management**: scripts para poblar entornos de test reproducibles y control de datos sensibles (anonimización).
- Incorporar **pruebas de seguridad** (SAST, DAST) y análisis de vulnerabilidades en pipeline.

Ejemplo: en una app de facturación para PYMEs, modelar BDD escenarios que incluyan reglas del DGI (por ejemplo: formato de timbrado) y ejecutar pruebas de aceptación automáticas que verifiquen cumplimiento normativo crítico antes de desplegar a producción.

3) Pipelines de CI/CD y quality gates (pipeline ejemplar)

Pipeline mínimo recomendado:

1. Commit / PR trigger

- Linting + format checks.
- Tests unitarios rápidos.
- Análisis estático (SAST) básico.
- Resultado: build *failed* si tests o SAST crítico falla.

2. Merge to develop

- Tests unitarios + tests de integración.
- Ejecutar análisis de cobertura.
- Ejecutar security scan (dependabot, scanning de dependencias).
- Generar artefacto y desplegar a staging si pasa quality gate.

3. Staging / Pre-release

- Tests e2e en entorno controlado (smoke tests).
- Performance/Load tests (si aplica) en night runs.

- DAST / pentest scripts (automatizados) en nightly.

4. Release / Production

- Despliegue automatizado con rollback.
- Monitoreo post-deploy y canary/gradual rollout si es posible.

Quality gates (condiciones para permitir merge/despliegue):

- Tests unitarios OK; coverage mínimo mantenido.
- No vulnerabilidades críticas por SAST/DAST.
- Code review aprobado.
- Build estable (no flakey failures).
- Resultado de smoke tests en staging OK.

Herramientas sugeridas (ejemplos): GitHub Actions / GitLab CI / Jenkins + SonarQube (estático), OWASP Dependency-Check, Snyk, JUnit/PyTest/Mocha, Selenium/Playwright (e2e), JMeter/k6 (performance). En entornos con recursos limitados, usar runners compartidos y pipelines optimizados por caché.

4) Prácticas de ingeniería y control de deuda técnica

Prácticas clave:

- **Code review** con checklist (ver más abajo).
- **Pair programming** en tareas complejas o críticas.
- **Refactoring continuo** y políticas de deuda técnica (límites de deuda, sprint dedicado si excede umbral).
- **Documentación mínima obligatoria** para APIs y esquemas de datos.
- **Versionamiento semántico** y releases etiquetadas.
- **Feature toggles** para despliegues de características experimentales sin afectar usuarios.

Gestión de deuda técnica: medir deuda (p. ej., issues críticos en SonarQube), priorizar por impacto y coste de corrección, reservar cada sprint un % de capacidad (ej. 10–20%) para deuda técnica en equipos estables.

5) Métricas y KPIs de calidad (definiciones, fórmulas y umbrales sugeridos)

A continuación un conjunto acotado de indicadores que proveen visión equilibrada entre producto y proceso:

1. Defect Density (defectos por KLOC)

- Fórmula: $\text{Defect Density} = (\text{Número de defectos encontrados}) / (\text{KLOC})$
- Interpretación: compara por tamaño de código.
- Umbral sugerido: depende del dominio; comparar con líneas base históricas.
- Ejemplo numérico (cálculo paso a paso): si en un módulo se encontraron 3 defectos y el módulo tiene 15 KLOC, entonces $\text{Defect Density} = 3 / 15 = 0.2$ defectos por KLOC. (Nota: multiplicar por 1000 es opción para expresar por 1000 LOC.)

2. Escaped Defects (defectos escapados a producción)

- Fórmula: $\text{Escaped Defects Rate} = (\text{Defectos detectados en producción}) / (\text{Total defectos detectados en el periodo})$
- Objetivo: disminuir. Umbral inicial: < 10% idealmente.
- Ejemplo: si 2 defects en producción y 18 defectos totales $\rightarrow 2/18 = 11.1\%$.

3. MTTR — Mean Time To Recover / Repair

- Fórmula: $\text{MTTR} = \Sigma (\text{Tiempo para resolver cada incidente}) / \text{Nº incidentes}$
- Objetivo: mantener bajo (p. ej., < 24 h para incidentes críticos).

- Ejemplo: 3 incidentes resueltos en 2 h, 6 h y 10 h → $MTTR = (2+6+10)/3 = 18/3 = 6 \text{ h.}$

4. **Coverage (cobertura de tests)**

- Métrica: % de líneas/branches/funciones cubiertas por pruebas.
- Umbral sugerido: *línea* $\geq 70\%$; *branch* $\geq 60\%$ para proyectos nuevos, ajustable según criticidad.

5. **Build Success Rate / Flakiness**

- Fórmula: $\text{Build Success Rate} = (\text{Nº builds exitosos}) / (\text{Nº builds total})$
- Objetivo: alta ($> 95\%$). Si baja, investigar flakiness tests.

6. **Time to Merge / Lead Time for Changes**

- Mide agilidad operativa; medir tiempo desde PR abierto hasta merge en main.

7. **SLA de disponibilidad (para sistemas críticos)**

- Ejemplo: 99.9% uptime anual.

8. **Satisfacción del stakeholder / NPS interno**

- Encuestas post-review 1–5.

Recomendación: elegir 4–6 KPIs que el equipo pueda medir sin alta carga administrativa y revisarlos en la Sprint Retrospective.

6) Checklists y plantillas prácticas

Checklist rápida de control de calidad por historia (para pegar en PR)

- Historia con criterios de aceptación claros.
- Tests unitarios que cubren los casos nuevos y edge-cases.
- Tests de integración para dependencias externas.
- Coverage no disminuye respecto al main branch más de X% (configurable).
- Code review aprobado por al menos 1 desarrollador distinto del autor.
- Linting y formateo OK.
- No hay vulnerabilidades nuevas en dependencias (report).
- Documentación actualizada (endpoints, migraciones).
- DoD completo.

Checklist para quality gate en CI

- Tests unitarios: pasar.
- Análisis estático: sin issues críticos.
- Security scan: no vulnerabilities critical/high.
- Coverage threshold: cumplido.
- PR reviews: aprobados.
- Artefacto creado: yes.

7) Code review — checklist (con criterios concretos)

- **Funcionalidad:** el cambio implementa la historia y pasa criterios de aceptación.
- **Claridad:** código legible; nombres descriptivos.
- **Complejidad:** evitar funciones/métodos > 150 líneas; sugerir refactor si complejo.
- **Pruebas:** existen tests; cubren casos límites.
- **Documentación:** comments cuando la lógica no es trivially understandable; actualizar README si modifica interfaces.

- **Seguridad:** evitar sanitización insuficiente; revisar entradas externas.
- **Performance:** identificar operaciones costosas en bucles críticos.
- **Compatibilidad:** no romper contrato público (APIs).

8) Matriz de riesgos de calidad y mitigaciones (ejemplo)

<i>Riesgo</i>	<i>Impacto</i>	<i>Probabilidad</i>	<i>Mitigación</i>
<i>Falta de adopción de pruebas automatizadas</i>	Alto	Media	Capacitación TDD; incluir tests como requisito de DoD
<i>Flaky tests que bloquean CI</i>	Medio	Alta	Clasificar y aislar tests inestables; separar en suite nightly
<i>Vulnerabilidades en dependencias</i>	Alto	Media	Escaneo automático; política de actualización semestral
<i>Falta de capacidad para soporte post-release</i>	Alto	Baja-Media	Plan de soporte y SLA con proveedor; rotación de on-call
<i>Problemas de rendimiento en horarios de matrícula</i>	Alto (UPN)	Media	Performance tests en staging; escalado automático

9) Actividad práctica (taller) — “Construye el Quality Plan y ejecútalo en 1 sprint”

Duración sugerida: 1–2 sesiones (4–6 horas en total).

Objetivos: diseñar Definition of Done, configurar pipeline mínimo (o simularlo), ejecutar tests y producir un informe de calidad.

Pasos:

1. *Diseño del Quality Plan* (60–90 min): equipo define DoD, KPIs, strategy tests, quality gates.
2. *Configuración/Simulación de pipeline* (60–90 min): crear scripts de tests unitarios y un job simple de CI (GitHub Actions/GitLab). Si no es posible, simular pasos manualmente y documentar.
3. *Ejecución y evidencia* (60–90 min): ejecutar suite de tests, generar reportes (coverage, pruebas, SAST).

4. *Análisis* (45–60 min): calcular KPIs y proponer 3 acciones de mejora priorizadas.
5. *Entrega*: Quality Plan + reportes + 3 acciones concretas.

Entregable esperado: Quality Plan (2–4 páginas) + pipeline minimal (o scripts) + informe de métricas y propuestas.

10) Ejemplo numérico breve (cálculo de MTTR y Escaped Defects)

Supón que en el último mes:

- Incidentes en producción: 4 (duraciones: 2 h, 8 h, 5 h, 1 h).
- Defectos detectados en total (incluido testing y producción): 20, de los cuales 4 en producción.

MTTR = $(2 + 8 + 5 + 1) / 4 = 16 / 4 = 4$ horas.

Escaped Defects Rate = $4 / 20 = 0.2 = 20\%$.

Interpretación: MTTR = 4 h (aceptable si SLA para incidentes críticos ≤ 8 h); escaped defects 20% → implicaría investigar cómo mejorar pipeline de pruebas y revisiones, apuntar a $< 10\%$ como meta

11) Consideraciones especiales para Nicaragua (aplicación práctica)

1. **Limitaciones de infraestructura:** en proyectos con conectividad limitada (zonas rurales), garantizar pruebas offline-first y validación en campo; automatización nocturna cuando haya conectividad.
2. **Capacitación y cultura:** invertir en capacitación práctica (TDD, code review) para equipos pequeños y mezclados (desarrolladores + operadores locales).

3. **Regulaciones:** en sistemas que impactan facturación o datos personales, integrar pruebas de cumplimiento normativo desde inicio (p.ej., formatos DGI, protección de datos).
4. **Recursos humanos:** equipos pequeños requieren perfiles T-shaped; priorizar pair programming y revisiones cruzadas para mantener calidad sin personal especializado para cada función.
5. **Sostenibilidad:** diseñar procesos de soporte local (capacitación a “champions” en cooperativas/universidad) para resolver incidentes sin depender siempre del proveedor externo.

12) Recomendaciones finales y hoja de ruta práctica

1. **Adoptar un Quality Plan mínimo** desde el primer sprint (DoD + pipeline simple).
2. **Shift-left:** incorporar pruebas y revisiones desde la primera línea de código.
3. **Automatizar lo crítico:** unit tests y análisis estático en cada commit; tests de integración en merge; e2e y performance en nightly.
4. **Medir poco y relevante:** 4–6 KPIs útiles y revisarlos en retrospectives.
5. **Plan de deuda técnica:** reservar capacidad en cada sprint (10–20%) para deuda técnica.
6. **Piloto:** ejecutar un piloto de 3 sprints para validar métricas y ajustar thresholds (p. ej., cobertura, MTTR).
7. **Formación continua:** talleres prácticos (TDD, code reviews, seguridad básica) para PO/SM/Dev.

Cambio

Scrum facilita el cambio (es empírico), pero **no significa “aceptar todo” sin control**. La gestión del cambio (Change Management) en proyectos ágiles equilibra:

- la **capacidad** del equipo (evitar sobrecarga y pérdida de foco),
- la **prioridad de valor** (que los cambios de alto valor se prioricen), y
- el **riesgo** (impacto técnico, regulatorio o de negocio).

La falta de un proceso claro conduce a *scope creep*, pérdida de calidad o frustración del equipo. En entornos como organismos públicos o cooperativas en Nicaragua, donde contratos, presupuesto y cumplimiento regulatorio son rígidos, un proceso formal pero ágil es crítico.

Principios operativos para la gestión del cambio en Scrum

1. **Centralizar la iniciativa, descentralizar la decisión:** cualquier stakeholder puede proponer un Change Request (CR), pero la decisión final sobre prioridad y tiempo la toma el Product Owner (con criterio técnico y con soporte del equipo).
2. **Transparencia completa:** todo CR registrado públicamente en el backlog o en un tablero de cambios.
3. **Evaluación rápida y empírica:** estimación rápida (t-shirt sizing o story points), análisis de riesgo y propuesta de respuesta (aceptar, posponer, derivar a proyecto paralelo).
4. **Adaptación controlada:** si se acepta un cambio, reflejarlo explícitamente en el siguiente Sprint Planning o crear un release backlog con su priorización.
5. **Protección del equipo:** evitar interrupciones frecuentes; establecer reglas (p. ej., cambios emergentes que justifiquen ruptura de sprint deben pasar por comité de emergencia).

Proceso recomendado (flujo) — versión corta y operativa

1. **Recepción:** cualquier stakeholder llena CR (plantilla).
2. **Registro:** CR registrado en herramienta (Backlog/Issue tracker) con ID.
3. **Triage rápido (48 h):** SM + PO revisan para clasificación (emergente/alto/medio/bajo).
4. **Estimación preliminar:** equipo asigna sizing (T-shirt / story points) en sesión corta (15–30 min).
5. **Análisis de impacto:** incluir impacto técnico, legal, financiero y en calidad (DoD).
6. **Decisión:** PO decide (aceptar/posponer/rechazar) y documenta razón y acción (si acepta: dónde entra en backlog).
7. **Implementación:** si aceptado, incluir en Sprint Planning o crear epic/issue para release futuro.
8. **Verificación:** al entregar, validar criterios de aceptación y DoD; registrar lecciones aprendidas.

Nota práctica: para cambios críticos fuera de sprint (p. ej., cumplimiento legal inminente), crear un “Comité de Emergencia” con representación de PO, SM, líder técnico y sponsor, que pueda autorizar excepciones con registro y retroalimentación en la siguiente retrospectiva.

Plantilla: Solicitud de Cambio (Change Request — CR) [texto para copiar]

CR-ID:

Fecha:

Solicitante: (nombre, rol, organización)

Título breve:

Descripción del cambio (qué y por qué):

Valor esperado / beneficio: (cualitativo y cuantitativo si aplica)

Urgencia / plazo requerido:

Categoría: (Regulatorio / Negocio / Técnico / Mejora / Bug)

Estimación preliminar: (T-shirt: S/M/L/XL; o story points)

Impacto en sprint/release: (ej.: requiere interrupción de sprint? afecta X historias?)

Impacto técnico: (dependencias, deuda, riesgo)

Impacto regulatorio o contractual:

Costos estimados (USD o C\$): (desarrollo one-time, operativos)

Recomendación inicial (PO/SM): (aceptar/posponer/rechazar)

Decisión (firma PO/Comité): (fecha + motivos)

Acción tomada: (ej.: añadida a backlog, generada epic, pospuesta)

Evidencias / notas post-implementación: (resultado del test, aceptación PO)

Criterios de admisibilidad (para que un CR sea considerado)

- CR tiene **propósito claro** y beneficio identificado.
- CR viene con **estimación preliminar** (o compromiso de estimarla en 48 h).
- CR no vulnera contratos o regulaciones sin plan de mitigación.
- Recursos disponibles o plan de financiación identificable.
- Para cambios fuera de sprint: justificación de emergencia documentada (p. ej., riesgo legal inmediato).

Cálculo de impacto (ejemplo numérico, paso a paso — evita errores aritméticos)

Situación: equipo Scrum con sprint de 2 semanas (10 días laborables). Su **velocidad** media = **30 story points / sprint**. Llega un CR estimado en **5 story points**. Queremos calcular el tiempo estimado en días laborables que equivalen esos 5 story points.

Cálculos (paso a paso):

1. Sprint length = 10 días.
2. Velocidad del equipo = 30 puntos por sprint.
3. Por tanto, puntos por día = velocidad / sprint_length = $30 / 10 = 3$ puntos / día.
4. Tiempo equivalente de 1 punto = $1 / (\text{puntos por día}) = 1 / 3 = 0.333333...$ días.
 - Si lo convertimos a horas (asumiendo 8 h/día): $0.333333... * 8 = 2.666666...$ horas por punto.
5. Para **5 puntos**: tiempo en días = $5 * 0.333333... = 1.666666... \text{ días}$.
6. Tiempo en horas = $1.666666... * 8 = 13.333333... \text{ horas}$.

Resultado interpretativo: un CR de 5 story points equivale aproximadamente a **1.67 días laborables** (~13.33 horas) de capacidad del equipo.

- Si el sprint ya está comprometido en un 90% de su capacidad, incorporar ese CR puede requerir re-priorizar y sacar ~5 puntos del sprint actual o mover el CR al siguiente sprint.

Opciones de respuesta ante un CR aceptado

1. **Incluir en el próximo Sprint Planning** (si no es urgente).
2. **Re-planificar el Sprint actual:** reemplazar ítems por el CR (si PO lo decide y el equipo acepta).
3. **Crear una Hotfix/Release:** para cambios críticos que no pueden esperar (ej., cumplimiento).
4. **Derivar a backlog de mantenimiento o epic:** si el cambio es grande, convertirlo en epic y planificar releases.
5. **Negociar alcance:** reducir scope del CR (MVP) para disminuir impacto inmediato.

Gobernanza y roles en la gestión del cambio

- **Product Owner:** decisión final sobre prioridad y trade-offs (valor vs coste).
- **Scrum Master:** facilita triage, asegura proceso y documenta impedimentos.
- **Equipo de Desarrollo:** estima impacto técnico y propone alternativas.
- **Sponsor / Comité de Gobernanza:** interviene en casos de presupuesto, contratos o cambios de alto impacto.
- **Representante legal/contable:** revisa cambios con implicaciones regulatorias o financieras.

Indicadores y métricas útiles (KPI) para monitorear cambios

- **Tasa de CR por periodo** (CRs / mes).
- **% de CR aceptados vs propuestos.**
- **Churn del backlog:** % de historias re-estimadas o cambiadas por sprint.

- **Tiempo medio de procesamiento de CR** (desde registro hasta decisión).
- **Impacto en velocity**: desviación de velocity promedio tras incorporar CRs.
- **% de CR que causan regresiones/defectos** (calidad).
- **Satisfacción del solicitante** (post-entrega, escala 1–5).

Riesgos comunes y medidas de mitigación

Riesgo	Medida de mitigación
<i>Scope creep / acumulación de pequeños cambios</i>	Limitar CRs por sprint; reglas claras para cambios menores; revisión semanal.
<i>Interrupciones frecuentes que degradan productividad</i>	Política de emergencia; reservar % de capacidad para cambios (p.ej., 10%).
<i>Cambios regulatorios con poco tiempo</i>	Plan de contingencia legal y fondo de emergencia; cláusulas contractuales para entregas.
<i>Decisiones contradictorias entre stakeholders</i>	Comité de gobernanza con priorización y escalamiento.
<i>Falta de trazabilidad</i>	Registro obligatorio de CRs con ID y vínculo con backlog y commits (VCS).

Ejemplo — Cooperativa cafetalera que requiere modificación en flujo de recepción de sacos

Contexto: después de un piloto, la cooperativa solicita que el formulario de registro incluya campo “grado de humedad” y cálculo automático de penalizaciones por humedad. CR submitted.

Proceso sugerido:

1. **Registro CR** con justificación (cumplimiento de norma de exportación y reducción de pérdidas).
2. **Triage en 48 h**: PO + SM clasifican como **Alto** (regulatorio/negocio).
3. **Estimación preliminar**: equipo estima 3 story points (ej.: UI + validaciones + cálculo). Usando ejemplo aritmético anterior, 3 points $\approx$ 1 día ($30/10 \Rightarrow 3$ pts/day $\rightarrow 3$ pts = 1 day).

4. **Análisis de impacto:** requiere validación con contador y pruebas de integración (export report); riesgo medio por dato sensible (humedad).
5. **Decisión:** PO acepta, lo incorpora en el próximo sprint y prioriza sobre una historia de baja prioridad.
6. **Implementación & verificación:** test de aceptación con muestras reales y sign-off del representante de calidad de la cooperativa.
7. **Registro de lección aprendida:** en retro, documentar que faltó ese requerimiento en el POC y ajustar checklist de validación en campo.

Actividad práctica para aula/taller — “Simulación de gestión de cambios”

Duración: 90–120 minutos.

Objetivo: practicar recepción, triage, estimación y decisión de 3 CRs (uno operativo, uno regulatorio, uno de mejora).

Pasos resumidos:

1. Dividir clase en grupos (PO, SM, Dev team, stakeholders).
2. Cada stakeholder presenta 1 CR (3 min cada uno).
3. Triage en 20 min: registrar CR, estimar rápido (planning poker light), documentar impacto.
4. Decisión y planificación: PO decide, equipo acuerda incorporación o posposición.
5. Presentación de resultados y retro de proceso (20 min).

Entregable: 3 CRs registrados + decisión y justificación (máx. 1 página).

Checklist práctica de gestión de cambios (para pegar en el repositorio / tablero)

- CR registrado con ID y datos completos.
- Triage realizado (clasificación y estimación preliminar).
- Impacto técnico y regulatorio documentado.
- Decisión tomada y registrada por PO/Comité.
- Si aceptado, backlog actualizado (historia/epic creada y estimada).
- Si afectó sprint en curso, equipo y stakeholders notificados y reasignación documentada.
- Verificación/QA completada y sign-off por PO.
- Lección aprendida registrada en retrospectiva.

Recomendaciones prácticas y consideraciones del contexto nicaragüense

1. **Contratos y compras públicas:** negociar cláusulas contractuales que permitan iteraciones y cambios por sprint; incluir CR como mecanismo formal de ajuste.
2. **Baja conectividad y validación en campo:** para cambios relacionados con trabajo en campo (cooperativas), exigir evidencia empírica (fotos, logs offline) antes de cerrar CR.
3. **Capacitación local:** formar “champions” en campo para identificar cambios reales vs deseos; esto reduce CRs frívolos.
4. **Financiamiento escalonado:** si un CR implica coste significativo, proponer financiamiento por hitos o piloto (reduce riesgo para sponsors).
5. **Marco regulatorio:** en proyectos con impacto legal (facturación, salud), involucrar desde el triage a representante legal o contador.

Riesgo

Scrum ayuda a **reducir** incertidumbre mediante inspección y adaptación, pero no elimina la necesidad de gestión de riesgos. La gestión proactiva de riesgos:

- minimiza sorpresas costosas,
- protege la capacidad del equipo (evita interrupciones frecuentes),
- mejora la toma de decisiones del Product Owner (PO) al priorizar backlog por riesgo/valor, y
- facilita cumplimiento regulatorio y continuidad operativa en contextos con limitaciones (p. ej., conectividad rural en Nicaragua).

La gestión de riesgos en entornos ágiles debe ser **ligera, continua y empírica**: detectar temprano + responder con acciones pequeñas y medibles.

2. Taxonomía de riesgos (clasificación práctica)

Clasificar ayuda a detectar patrones y a elegir respuestas apropiadas.

A. Técnicos

- Deuda técnica acumulada
- Integración fallida / builds rotos
- Falta de pruebas automatizadas / flakey tests

B. Organizacionales / Humanos

- Rotación de personal clave (PO, SM, Dev)
- Falta de disponibilidad del PO
- Resistencia al cambio en la organización

C. De negocio / financieras

- Presupuesto insuficiente
- Cambios regulatorios que afectan alcance (ej.: DGI)
- Mercado/cliente que cambia requisitos drásticamente

D. Operativos / Infraestructura

- Fallas en hosting o proveedor cloud
- Conectividad limitada en zonas rurales
- Dependencias externas (APIs de terceros)

E. Legales / Compliance

- Requisitos legales no cumplidos (facturación, datos personales)
- Contratos que no permiten entregas iterativas

F. Externos / Naturales

- Eventos climáticos (huracanes), disturbios, pandemias
- Problemas logísticos (transporte de hardware)

3. Proceso operativo de gestión de riesgos (adaptado a Scrum)

Este proceso es iterativo y se integra en la cadencia Scrum:

1. Identificación continua (quincenal / ongoing)

- Quién: todo el equipo + stakeholders.
- Cuándo: backlog refinement, sprint planning, y por triggers ad-hoc.
- Artefacto: **Risk Register** (ver plantilla abajo).

2. Análisis

- **Cualitativo:** probabilidad (Baja/Media/Alta) × impacto (Bajo/Medio/Alto) → prioridad.
- **Cuantitativo** (cuando procede): $EMV = Probabilidad \times Impacto$ monetario. Usar para riesgos con impacto financiero relevante.
- Herramientas: matriz de calor, árbol de decisión, análisis de sensibilidad.

3. Respuesta / Tratamiento

- Opciones: **Evitar, Mitigar, Transferir** (seguros, contratos), **Aceptar** (con contingencia).

- Definir acciones, responsables, recursos y triggers (umbrales que activan la respuesta).

4. Monitoreo y control

- Revisar riesgos en cada daily/weekly impediments list, incluir en sprint backlog si hay acciones.
- Reporte en Sprint Review y ajustes en Retrospective.
- KPIs de seguimiento (ver sección KPIs).

5. Lecciones aprendidas

- Documentar en retrospective + actualizar el registro y thresholds.

4. Herramientas y artefactos (plantillas listas para usar)

Plantilla: Registro de riesgos (Risk Register) — columnas recomendadas

- **ID**
- **Título del riesgo**
- **Descripción**
- **Categoría** (Técnico/Organizacional/Legal/Externo/Financiero/Operativo)
- **Probabilidad** (Baja = 10%, Media = 50%, Alta = 90% — o escala 1–5)
- **Impacto (USD o C\$)** (estimación monetaria directa o costo equivalente)
- **EMV (USD)** = Probabilidad × Impacto (ver ejemplo numérico)
- **Prioridad** (alto/medio/bajo) — derivada de matriz probabilidad×impacto o EMV
- **Respuesta propuesta** (Evitar/Mitigar/Transferir/Aceptar)
- **Acciones específicas** (qué se hará)
- **Responsable**
- **Trigger / Umbral** (qué activa la acción)
- **Estado** (abierto/en progreso/mitigado/cerrado)
- **Fecha de revisión**
- **Notas / Evidencias**

Puedes implementar esto como hoja de cálculo (Google Sheets / Excel) o como issue type en Jira/Redmine.

5. Análisis cuantitativo: Expected Monetary Value (EMV) — ejemplo paso a paso

El **EMV** ayuda a priorizar riesgos por exposición monetaria. Presento un ejemplo típico y su cálculo digitado paso a paso (siguiendo la regla de revisar aritmética):

Riesgo A: Robo de hardware en operaciones en campo.

- Probabilidad estimada anual: **10%** → como decimal = **0.10**.
- Impacto directo estimado (reemplazo hardware + pérdida de datos y horas de trabajo): **USD 2,400**.

Cálculo EMV (paso a paso):

1. Probabilidad en decimal = 10% = 0.10.
2. Impacto = 2,400 USD.
3. $EMV = Probabilidad \times Impacto = 0.10 \times 2,400$.

Ahora multiplicamos con cuidado:

- $2,400 \times 0.10 = (2,400 \times 10) / 100 = 24,000 / 100$.
- $24,000 / 100 = 240$.

Resultado: $EMV = \text{USD } 240$.

Interpretación: el “coste esperado anual” de ese riesgo es USD 240; si multiplicas por varios riesgos obtendrás la exposición anual esperada.

Ejemplo con varios riesgos (sumario EMV)

Supón tres riesgos para una cooperativa:

1. **R1 — Robo de hardware:** Prob = 10% (0.10); Impacto = USD 2,400 →
EMV = USD 240 (como calculado).
2. **R2 — Pérdida de conectividad durante cosecha (impacto operativo):**
Prob = 30% → 0.30; Impacto estimado (pérdida de ventas/eficiencia) = USD 1,500.
 - $EMV = 0.30 \times 1,500$.
 - $1,500 \times 0.30 = (1,500 \times 30) / 100 = 45,000 / 100 = 450$.
 - EMV = **USD 450**.
3. **R3 — Cambio regulatorio (DGI) obligando a rework:** Prob = 5% → 0.05; Impacto = USD 6,000.
 - $EMV = 0.05 \times 6,000$.
 - $6,000 \times 0.05 = (6,000 \times 5) / 100 = 30,000 / 100 = 300$.
 - EMV = **USD 300**.

Exposición anual total (suma de EMV) = 240 + 450 + 300 = USD 990.

(Operación paso a paso: $240 + 450 = 690$; $690 + 300 = 990$.)

Interpretación práctica: la cooperativa puede pensar en un fondo anual de mitigación o seguro aproximado a USD 990 (o priorizar mitigación para los riesgos con mayor EMV, en este caso R2).

6. Matriz de calor (heatmap) y umbrales sugeridos

Se construye cruzando **probabilidad** (Baja/Media/Alta) con **impacto** (Bajo/Medio/Alto). Ejemplo de umbrales:

- **Rojo (Alto):** Probabilidad Alta × Impacto Alto — requiere acción inmediata (evitar/transferir/mitigar).
- **Naranja (Medio-Alto):** combina Alta×Medio o Media×Alto — plan de mitigación y contingencia.
- **Amarillo (Medio):** Media×Medio o Baja×Alto — monitor y medidas de mitigación moderadas.
- **Verde (Bajo):** Baja×Bajo, Baja×Medio — aceptar con monitoreo.

Recomendación de umbrales numéricos (ejemplo que puedes usar):

- Baja = 10% (0.10)
- Media = 30% (0.30)
- Alta = 60% (0.60)

Impacto monetario (umbral):

- Bajo < USD 1,000
- Medio = USD 1,000–5,000
- Alto > USD 5,000

Estos valores deben adaptarse al tamaño del proyecto/organización.

7. Estrategias de respuesta (con ejemplos prácticos)

- **Evitar:** eliminar la actividad que genera riesgo (ej.: no depender de un proveedor único para pagos; usar dos proveedores).
- **Mitigar:** reducir probabilidad o impacto (ej.: cifrado de datos, backups, offline-first design, formación).
- **Transferir:** pasar impacto a tercero (seguro, SLA con proveedor).
- **Aceptar:** decidir no tomar acción activa; establecer contingencia y fondos.

Ejemplo por estrategia:

- **Mitigar** riesgo de conectividad: diseño *offline-first* + compresión de imágenes + sincronización diferida; resultado: reduce probabilidad efectiva de fallo operativo del 30% a 10% (do the math if needed).
- **Transferir** riesgo de robo: contratar seguro contra robo para hardware en campo; costo anual del seguro debe compararse con EMV.
- **Evitar** riesgo regulatorio: consultar contador/legal antes de release crítico y establecer checklist de cumplimiento para historias que toquen facturación.

8. Integración operativa con Scrum (¿dónde viven los riesgos?)

- **Backlog Refinement:** registrar riesgos detectados como **issues** o **spikes** para estimación técnica y mitigación (p. ej., spike para evaluar alternativas de sincronización offline).
- **Sprint Planning:** considerar capacidad para acciones de mitigación; reservar porcentaje de sprint para mitigación si hay riesgos activos.
- **Daily Scrum:** reportar indicadores de riesgo y triggers (p. ej., "se detectó subida de errores en integración" → activar plan).
- **Sprint Review:** revisar riesgos materializados y ajustar prioridades con stakeholders.

- **Retrospective:** evaluar efectividad de las respuestas y ajustar thresholds/acciones.

Buenas prácticas: establecer un “Risk Backlog” o etiquetar riesgos en el Product Backlog; el PO prioriza mitigaciones por valor mitigado (EMV reducido).

9. Roles y responsabilidades

- **Product Owner:** priorizar mitigaciones en backlog, decidir transferencias/aceptaciones por valor.
- **Scrum Master:** facilitar identificación y escalamiento, asegurar que mitigaciones no se conviertan en impedimentos prolongados.
- **Equipo de Desarrollo:** estimar e implementar medidas técnicas de mitigación.
- **Sponsor / Dirección:** aprobar fondos para transferencias (seguros) o acciones costosas.
- **Responsable de Riesgos (opcional en proyectos grandes):** mantener Risk Register y reportar métricas.

10. KPIs y métricas de riesgos (qué medir)

Escoge 6–8 indicadores prácticos, por ejemplo:

1. **Exposición de riesgo total (sum EMV)** — USD (revisar trimestralmente).
2. **Número de riesgos en estado crítico (rojo)** — cuenta.
3. **Tiempo medio de respuesta a triggers** — horas/días.
4. **% mitigaciones implementadas en plazo** — control de ejecución.
5. **% de CR que derivaron en incidentes** — vinculación riesgo→evento.
6. **Costo real vs EMV estimado** — comparar expectativas vs realidad (aprendizaje).
7. **SLA de incidentes críticos (MTTR)** — horas.

8. **% de sprints con mitigaciones planificadas** — control de capacidad dedicada a riesgos.

11. Ejemplo completo — Caso: “Digitalización de inventario” para cooperativa en Matagalpa

Presento un conjunto de riesgos, análisis EMV, respuesta y plan de contingencia (resumen operativo).

RIESGOS (seleccionados)

1. R1 — Baja conectividad en campo

- Prob: 30% (0.30) — Media.
- Impacto estimado: USD 1,500 (eficiencia perdida, retrasos).
- $EMV = 0.30 \times 1,500 = 450$ USD (cálculo: $1,500 \times 0.30 = 45,000/100 = 450$).
- Respuesta: Mitigar → desarrollo offline-first + compresión + retries; responsable: equipo Dev; trigger: >3 reportes de sincronización fallida en 1 semana.
- Contingencia: proveer tablets con SIM/plan de datos para puntos críticos; presupuesto alternativo.

2. R2 — Robo/daño de tablets

- Prob: 10% (0.10). Impacto: USD 2,400 (replacement + data recovery). $EMV = 240$ USD.
- Respuesta: Transferir → seguro de equipo; Mitigar → gestión física (cierres, inventario) y cifrado; trigger: 1 incidente de robo en 6 meses.

3. R3 — Requisito regulatorio de exportación cambia (humedad/etiquetado)

- Prob: 5% (0.05). Impacto: USD 6,000 (rework + re-certificación). $EMV = 300$ USD.

- Respuesta: Mitigar/evitar → incluir validaciones y consulta legal; contingency: plan de rework priorizado como epic.

4. **R4 — Rotación de personal clave (operador de campo)**

- Prob: 20% (0.20). Impacto: USD 3,000 (costos de capacitación y retraining). EMV = 600 USD (cálculo: $3,000 \times 0.20 = 600$).
- Respuesta: Mitigar → capacitar “champions” locales y documentación exhaustiva; trigger: aviso de salida con <30 días.

Exposición total (sum EMV): $450 + 240 + 300 + 600 = \text{USD } 1,590$.

(Cálculo paso a paso: $450 + 240 = 690$; $690 + 300 = 990$; $990 + 600 = 1,590$.)

Priorización: R4 (EMV 600) y R1 (450) primero; R3 y R2 después o transferir R2 al seguro.

12. Plan de contingencia — ejemplo operativo para R1 (conectividad)

- **Trigger:** 3 fallos de sincronización reportados por usuarios en 7 días.
- **Acción inmediata (0–24 h):** notificación al SM y al responsable de infraestructura; activar canal alternativo (envío de fotos por WhatsApp como evidencia temporal).
- **Acción a corto plazo (1–3 días):** asignar un desarrollador para desplegar parche que mejore retries y compresión; notificar usuarios.
- **Acción a mediano plazo (1–4 semanas):** plan de mejora offline-first + test en campo; priorizar en sprint siguiente.
- **Indicador de éxito:** reducción de fallos de sincronización a <1 por semana en 2 semanas tras despliegue.

13. Actividad práctica (taller) — “Riesgo en acción”

Objetivo: identificar, analizar EMV y definir mitigaciones para 6 riesgos de un caso real.

Duración: 120–150 minutos.

Pasos resumidos:

1. Dividir grupos y asignar un caso (PYME, cooperativa, dependencia pública).
2. Identificación (20 min): lluvia de riesgos y categorización.
3. Análisis cualitativo y cuantitativo (40 min): estimar prob/impacto y calcular EMV (mostrar cálculos).
4. Diseñar respuestas y triggers (30 min).
5. Presentaciones (20–30 min) + retro.

Entregable: Risk Register con ≥ 6 riesgos, EMV calculado y plan de contingencia para los 3 de mayor EMV.

14. Buenas prácticas y recomendaciones finales (acción inmediata)

1. **Empieza pequeño y mejora:** mantén el Risk Register simple (hoja de cálculo) y evoluciona a herramientas integradas si el proyecto crece.
2. **EMV para decisiones importantes:** calcula EMV cuando el impacto económico sea relevante; para riesgos operativos, usa métricas de MTTR y frecuencia.
3. **Vincula riesgos al backlog:** mitigaciones importantes deben convertirse en historias técnicas o spikes y priorizarse por el PO.
4. **Reserva capacidad:** considera reservar 5–15% de la capacidad del sprint para mitigación si la exposición es alta.
5. **Automatiza la detección:** logs, alertas y dashboards para triggers críticos (p. ej., tasas de error, fallos de build).
6. **Revisa regularmente:** integra revisión de riesgos en Retrospective y Sprint Review.

7. **Documenta lecciones aprendidas:** actualiza probabilidades/impactos con evidencia real (aprendizaje empírico).

Inicio

Aunque Scrum promueve iteraciones rápidas, **el inicio del proyecto** (project initiation) es la fase donde se establecen los fundamentos: propósito, alcance mínimo viable, stakeholders clave, estructura del equipo y las reglas básicas de interacción. Un buen inicio no pretende detallar todo, sino reducir incertidumbres críticas para permitir que los Sprints empiecen con prioridades claras y una gobernanza que permita adaptación. El empirismo (inspección → adaptación) opera mejor cuando se inicia con hipótesis explícitas y mecanismos de validación.

Artefactos esenciales del inicio (qué producir como mínimo)

1. **Project Charter (Carta del Proyecto)** — documento corto que responde: ¿por qué existe el proyecto?, ¿qué entregará?, ¿quién lo financia?, ¿cuáles son los principales riesgos y supuestos?, ¿qué criterios de éxito?.
2. **Stakeholder Register** — lista priorizada de interesados: rol, influencia, expectativas, canal y frecuencia de comunicación.
3. **Equipo y gobernanza** — organigrama, roles (PO, SM, Development Team), RACI para decisiones clave (priorización, cambios en alcance, aprobaciones presupuestarias).
4. **Product Backlog inicial (MVP)** — backlog mínimo priorizado con 6–12 user stories o epics para arrancar el primer sprint/piloto.
5. **Definition of Done (DoD) propuesta** — criterios mínimos para considerar un incremento “hecho”.
6. **Plan de comunicaciones** — quién informa a quién, con qué frecuencia y por qué medio (incluye plan de reporting para sponsors).
7. **Registro inicial de riesgos y supuestos** — 8–12 riesgos o supuestos con prioridad y acciones iniciales.

8. **Kickoff Meeting agenda y acta** — agenda detallada, asistentes, decisiones y próximos pasos.

Plantilla: Project Charter (texto para copiar / completar)

Título del proyecto:

Sponsor / Financiador:

Product Owner (temporal):

Scrum Master (temporal):

Fecha de inicio propuesta:

Visión / Propósito (1–3 frases): — ¿qué problema resuelve y para quién?

Objetivos SMART del proyecto (máx. 3): — ejemplo: “Reducir tiempo de registro de inventario en campo en 70% en 12 meses.”

Alcance (lo que entra / lo que no entra):

Entregables principales (MVP): — lista corta (3–5 ítems).

Supuestos críticos: — p. ej., disponibilidad del PO 3 días/semana; acceso a datos históricos.

Restricciones: — presupuesto, regulaciones, conectividad.

Criterios de éxito (KPIs iniciales): — p. ej., payback estimado, % reducción errores, satisfacción stakeholders.

Riesgos críticos iniciales (3–5): — enumerar.

Presupuesto inicial / financiación (estimación):

Cadencia propuesta de trabajo: sprint length (p. ej., 2 semanas), revisión con stakeholders (cada sprint).

Firma del sponsor / aceptación: (nombre, fecha)

Ejemplo práctico (resumido) — Cooperativa cafetalera (inicio del proyecto)

Título: Digitalización de inventario de sacos de café — Cooperativa “Cosecha Viva”.

Visión: Permitir registro y trazabilidad de sacos desde finca hasta bodega, reduciendo pérdidas.

MVP: registro móvil de ingreso de sacos con foto y QR; generación de reporte de lotes.

Objetivos SMART: reducir tiempo de registro en 70% en 6 meses; reducir errores de inventario en 50% en 12 meses.

Supuestos críticos: tablets proporcionadas por cooperativa; conectividad limitada (offline-first obligatorio).

Riesgos iniciales: baja conectividad, resistencia a uso de tablets, presupuesto limitado.

Criterios de éxito: adoption rate 70% de operadores en 3 meses; payback < 4 años (ver supuesto en justificación).

Kickoff Meeting — Agenda modelo (2 horas)

1. **Bienvenida y objetivos del Kickoff** (10 min) — sponsor / PO.
2. **Visión del proyecto y alcance (Charter)** (15 min) — PO.
3. **Stakeholders y beneficios esperados** (10 min).
4. **Equipo y gobernanza (roles y RACI)** (15 min) — SM.
5. **Product Backlog inicial y MVP** (20 min) — PO + equipo técnico.
6. **Definition of Done y criterios de calidad** (10 min).
7. **Riesgos y supuestos críticos** (15 min).
8. **Plan de comunicaciones y cadencia** (10 min).
9. **Próximos pasos y asignaciones** (10 min).
10. **Preguntas, compromisos y cierre** (5 min).

Acta de Kickoff: registrar asistentes, decisiones, acuerdos y próximos pasos (responsable y fecha).

Stakeholder Register — columnas recomendadas

- Nombre / Organización
- Rol / Interés
- Nivel de influencia (Alto/Medio/Bajo)
- Expectativas clave
- Canal preferido (email, reunión presencial, WhatsApp)
- Frecuencia de comunicación
- Acción de involucramiento (p.ej., demo técnica cada sprint)

Plan de comunicaciones (resumen práctico)

- **Sponsor / Dirección:** reporte ejecutivo semanal/bimensual por email y reunión mensual.
- **Stakeholders clave (clientes):** demo cada sprint (Sprint Review) + resumen por escrito.
- **Equipo interno:** daily scrum (15 min), canal de chat para impedimentos, tablero actualizado diario.
- **Usuarios en campo:** comunicación asíncrona (fotos, logs) + reuniones presenciales trimestrales.

Backlog inicial (MVP) — estructura sugerida

- ID | User Story (INVEST) | Prioridad | Estimación (T-shirt) | Criterios de aceptación | Dependencias

Ejemplo (para cooperativa):

- US-01 | Como operario quiero registrar entrada de saco con foto para identificar lote. | Alta | M | CA: foto adjunta, QR generado, registro sincronizado.
- US-02 | Como administrador quiero generar reporte por lote por fecha. | Alta | M | CA: export CSV, filtros por finca.
(6–12 stories es suficiente para arrancar un piloto).

Checklist de inicio

Antes del Kickoff

- Project Charter completado y aprobado por sponsor.
- Stakeholder Register inicial creado.
- Equipo mínimo identificado (PO, SM temporal, Dev Team provisional).
- Backlog inicial (MVP) con 6–12 historias.
- DoD propuesta preparada.
- Registro inicial de riesgos y supuestos.
- Plan de comunicaciones preliminar.

Durante/Al cierre del Kickoff

- Acta de kickoff firmada con próximos pasos y responsables.
- Primera fecha de Sprint Planning establecida.
- Espacio/Canal de comunicación definido y probado (Slack/WhatsApp + tablero).
- Recursos (hardware, acceso a datos) confirmados o plan de adquisición.
- Fecha y responsables para la POC o sprint piloto definidos.

Actividad práctica para aula / taller — “Kickoff simulado”

Duración: 2–3 horas.

1. Divide clase en grupos (PYME / cooperativa / universidad).
2. Cada grupo completa el Project Charter (30–45 min).
3. Preparan y simulan un Kickoff (20–30 min de presentación + 10 min Q&A).
4. Entregan acta y Backlog inicial.
5. Feedback grupal y rúbrica de evaluación.

Entregable: Charter, Backlog (6–12 stories), acta de kickoff.

Rúbrica de evaluación (Project Charter + Kickoff) — total 12 pts

- 1. Claridad de visión y objetivos SMART (0–4)**
 - 0–1: visión vaga; objetivos no medibles.
 - 2: visión presente, objetivos parcialmente medibles.
 - 3: visión clara; objetivos medibles pero poco concretos.
 - 4: visión y 2–3 objetivos SMART claros.
- 2. Backlog MVP y criterios de aceptación (0–4)**
 - 0–1: sin historias o mal formuladas.
 - 2: historias presentes pero sin criterios claros.
 - 3: historias bien formuladas; criterios de aceptación en la mayoría.
 - 4: backlog MVP coherente con visión y criterios completos.
- 3. Stakeholders, gobernanza y plan de comunicaciones (0–4)**
 - 0–1: falta identificación o plan.
 - 2: stakeholders identificados; gobernanza incompleta.
 - 3: buena definición; plan de comunicaciones básico.
 - 4: stakeholders priorizados, RACI y plan de comunicación claros y adecuados al contexto.

Interpretación: ≥ 10 = inicio sólido; 7–9 = aceptable; < 7 = requiere revisión.

Consideraciones prácticas y adaptaciones al contexto nicaragüense

- **Procurement y presupuestos públicos:** si trabajas con organismos, documenta cláusulas contractuales que permitan iteraciones (p. ej., entregables por sprint) y estipula criterios de aceptación vinculantes.
- **Baja conectividad:** planifica MVP offline-first y arrange acceso alternativo (p. ej., sincronización por USB o puntos periódicos con Internet).
- **Capacitación y adopción:** asigna “champions” locales y plan de formación en kickoff; en cooperativas, incluye sesiones presenciales de demostración.
- **Financiamiento escalonado:** proponer liberación de fondos por hitos (ej.: 30% inicio, 40% tras POC, 30% post-aceptación) reduce riesgo financiero.
- **Sensibilidad cultural:** adaptar lenguaje, horarios y protocolos de reunión a prácticas locales (p. ej., coordinar reuniones presenciales en días de mercado con antelación).

Un **inicio** bien ejecutado no pretende eliminar la incertidumbre —eso es imposible— sino *minimizar las incertidumbres que impiden comenzar con sentido y foco*. Si sigues las plantillas y checklists aquí presentadas, tendrás un arranque ordenado, defendible ante sponsors y con mecanismos claros para inspeccionar y adaptar según avance el proyecto.

Planificación y estimación

- **Estimación relativa (no predictiva absoluta):** las estimaciones en Scrum sirven para *comparar* esfuerzo/valor entre items y para *planificar capacidad*, no para prometer horas exactas a stakeholders no técnicos.
- **Planificación empírica:** se usa historial (velocity) y observación para refinar pronósticos; la precisión mejora con datos reales (aprendizaje iterativo).
- **Propósito práctico:** priorizar el Product Backlog, decidir alcance del sprint, gestionar expectativas de stakeholders, y proyectar releases.

Fundamento académico: estudios en ingeniería de software indican que la estimación relativa (story points) reduce sesgos y mejora la precisión de predicción respecto a estimaciones en horas absolutas, especialmente cuando se trabaja con equipos estables y con historial. (La clave es la calibración y la revisión periódica).

2. Historias de usuario: redacción y criterios de aceptación

INVEST (regla mnemotécnica)

Cada historia debería cumplir:

- **I**ndependent (Independiente)
- **N**egotiable (Negociable)
- **V**aluable (Valiosa para el usuario/negocio)
- **E**stimable (Estimable)
- **S**mall (Pequeña — que entre en un sprint)
- **T**estable (Con criterios de aceptación)

Plantilla estándar

Como [rol]

quiero [funcionalidad]

para [beneficio].

Criterios de aceptación:

- Dado [condición], cuando [acción], entonces [resultado].
- ...

Notas/Dependencias:

- ...

Ejemplo (cooperativa cafetalera — US realista)

US-01:

Como operario de finca

quiero registrar la entrada de un saco con foto y código QR para garantizar la trazabilidad del lote.

Criterios de aceptación:

- Dado que el operario toma la foto, cuando guarda el registro, entonces se genera un QR único y el registro queda disponible para sincronización.
- El formulario valida que el peso esté entre 30 kg y 70 kg.
- El Product Owner acepta la demo con 3 registros de prueba.

3. Técnicas de estimación — descripción y protocolo

a) Estimación relativa con Story Points (principios)

- Los *story points* miden **complejidad relativa**, esfuerzo y riesgo combinados, no horas absolutas.
- Ventaja: reducen discusión sobre horas y ponen foco en comparación.
- Equipo debe calibrar un *benchmark story* (ej.: “1 punto” o “3 puntos”) como referencia.

b) Planning Poker (protocolo)

1. Seleccionar 5–10 historias para sesión.

2. Cada participante tiene cartas con secuencia (Fibonacci adaptada): 0, 1, 2, 3, 5, 8, 13, 20, 40, 100.
3. Se lee la historia; breve clarificación si hace falta (no discusión larga).
4. Cada miembro selecciona carta en privado y la muestra simultáneamente.
5. Si hay consenso (misma carta) se asigna ese punto; si no, los de mayor y menor justifican, se discute 2–3 minutos y se repite hasta consenso o segunda votación.
6. Registrar estimación y mover a siguiente historia.

Duración: 20–40 historias en 60–90 minutos con equipo entrenado.

c) Affinity Estimation (para grandes backlogs)

- Ordenar historias en una pared o tablero por tamaño relativo en grupos (S/M/L/XL) de manera colaborativa y luego asignar números (puntos) a cada grupo.
- Es eficiente para estimar muchos ítems rápidamente.

d) T-shirt sizing (S/M/L/XL)

- Útil en fases tempranas o para stakeholders no técnicos; convertir luego a puntos relativos.

e) Spikes (investigación técnica)

- Si una historia es incierta, crear un *spike* (tarea de tiempo limitado, p. ej., 2–3 días) para investigar y retornar con estimación más precisa.

4. De puntos a capacidad — cómo convertir y planificar (con ejemplo aritmético paso a paso)

Importante: no existe una “tasa universal” puntos→horas. Cada equipo debe **calibrar**. A continuación muestro un método práctico y ejemplos numéricos calculados paso a paso para que el procedimiento sea reproducible y evite errores aritméticos.

Paso A — Calcular velocidad histórica (si existe)

Supón que el equipo registró las siguientes entregas en 4 sprints:

- Sprint 1 = 28 puntos
- Sprint 2 = 32 puntos
- Sprint 3 = 30 puntos
- Sprint 4 = 30 puntos

Velocidad promedio (Vprom):

1. Sumar: $28 + 32 + 30 + 30$.
 - $28 + 32 = 60$.
 - $60 + 30 = 90$.
 - $90 + 30 = 120$.
2. Dividir por número de sprints (4): $120 / 4$.
 - 120 dividido entre 4 = 30.

Resultado: Vprom = **30 puntos por sprint**.

Paso B — Calcular puntos por día (equivalencia operativa)

Si sprint length = 2 semanas = 10 días laborables (ajusta según tu calendario):

1. Puntos por día = $V_{prom} / \text{sprint_days} = 30 / 10$.
 - 30 dividido entre 10 = 3.

Resultado: **3 puntos / día**.

Paso C — Convertir story points a días y horas (ejemplo para una historia de 5 puntos)

1. Días por punto = $1 / (\text{puntos por día}) = 1 / 3$.
 - 1 dividido entre 3 = 0.333333... (repetido).
 - Para trabajo práctico, redondeamos a **0.3333 días / punto** si se desea.
2. Para 5 puntos: días = $5 * 0.333333...$
 - $5 \times 0.333333... = 1.666666... \text{ días}$.
3. Convertir a horas (si jornada = 8 h/día): horas = $1.666666... * 8$.
 - $1.666666... \times 8 = 13.333333... \text{ horas}$.

Interpretación: 5 puntos $\approx$ **1.67 días** $\approx$ **13.33 horas**.

5. Planificación de sprint — protocolo práctico y agenda

Objetivo del Sprint Planning

Decidir **qué** se entregará en el sprint (Sprint Backlog) y **cómo** el equipo planea conseguirlo (plan de trabajo), con un Sprint Goal claro.

Agenda sugerida (2 horas para sprint de 2 semanas)

1. **Revisión del Sprint Goal anterior y datos (10–15 min)** — SM/PO.
2. **PO presenta backlog priorizado y justifica prioridades (15–20 min)**.
3. **Equipo estima/valida los items seleccionados y confirma capacidad (20–30 min)**.
4. **Descomposición en tareas y asignación inicial (30–40 min)**.
5. **Definición del Sprint Goal y confirmación del compromiso (10–15 min)**.
6. **Registro de riesgos e impedimentos previstos (10 min)**.

Salidas esperadas

- Sprint Goal (frase clara).
- Sprint Backlog con historias y tareas, estimaciones y dueños.
- Plan de mitigación para riesgos críticos asignados.
- Definition of Done confirmada para sprint.

6. Ejemplo de planificación y proyección de release (ejemplo completo)

Contexto (PYME de Managua)

- Backlog total priorizado (MVP) suma = **180 puntos**.
- Equipo: mismo del ejemplo anterior con **velocidad promedio = 30 puntos/sprint**.
- Sprint length = 2 semanas.

¿Cuántos sprints para completar el backlog?

1. Dividir total puntos por velocidad por sprint: $180 / 30$.
 - 180 dividido entre 30 = 6.

Resultado: 6 sprints.

Convertir sprints a tiempo calendario

- 6 sprints $\times$ 2 semanas/sprint = 12 semanas.
- 12 semanas $\approx$ 3 meses (si consideras 4 semanas por mes).

Añadir buffer conservador (p. ej., 20%) para imprevistos

1. Buffer en sprints = $6 \times 0.20 = 1.2$ sprints.
 - $6 \times 0.20 \rightarrow$ calcular: $6 \times 2 = 12$; luego $12 / 10 = 1.2$ (forma alternativa) — pero directamente $6 \times 0.20 = 1.2$.
2. Total sprints con buffer = $6 + 1.2 = 7.2$ sprints.
 - Redondear a 8 sprints si se requiere integer sprints.
3. Tiempo calendario ≈ 8 sprints $\times$ 2 semanas = 16 semanas $\approx$ 4 meses.

Interpretación: plan base = 6 sprints (3 meses); plan conservador con 20% buffer $\approx$ 8 sprints (4 meses).

7. Capacity planning

Elementos a considerar

- **Disponibilidad nominal** de cada miembro (días laborables del sprint).
- **Tiempo no productivo**: reuniones, formación, administración (ej.: 20–30% del tiempo).
- **Vacaciones / feriados / permisos**.
- **Porcentaje reservado para imprevistos / cambios urgentes** (p. ej., 10–15%).

Método práctico (ejemplo numérico paso a paso)

Equipo: 5 miembros dev (no incluye PO/SM para velocidad), jornada productiva estimada = 8 h/día.

Sprint: 10 días.

1. Horas brutas por miembro = $10 \text{ días} \times 8 \text{ h/día} = 80 \text{ h}$.
2. Horas totales brutas equipo = $5 \text{ miembros} \times 80 \text{ h}$.
 - $5 \times 80 = 400 \text{ h}$.
3. Descontar factor no productivo (reuniones, admin): asumimos 25% no productivo.
 - Horas productivas por miembro = $80 \text{ h} \times (1 - 0.25) = 80 \times 0.75$.
 - $80 \times 0.75 = 60 \text{ h}$.
4. Horas productivas totales equipo = 5×60 .
 - $5 \times 60 = 300 \text{ h}$.
5. Si equipo históricamente entrega 30 puntos/sprint (V_{prom}) y horas productivas totales = 300 h, podemos calcular **horas por punto**:
 - Horas por punto = $300 \text{ h} / 30 \text{ puntos}$.
 - 300 dividido entre 30 = 10 h/punto.

Resultado: 10 horas por punto (equivalencia del equipo bajo supuestos dados).

Interpretación práctica: si una historia es 5 puntos → tiempo estimado = 5×10 h = 50 h (esto debe concordar con días y tareas).

- Convertir a días laborales: $50 \text{ h} / 8 \text{ h/día} = 6.25$ días.

Nota: esta conversión es una herramienta para planificación inicial o cuando no hay histórico; una vez que exista velocity es preferible usarla. Siempre recalibrar al cabo de 2–3 sprints.

8. Manejo de incertidumbre y buffers (técnicas recomendadas)

- **Reserva de capacidad en sprint:** reservar entre 10–20% de la capacidad para emergencias/CRs.
- **Planificación por escenarios:** ofrecer al sponsor plan base (50% prob), plan optimista y plan conservador (incluir buffer del 20–30%).
- **Spikes:** usar para reducir incertidumbre antes de comprometer historias grandes.
- **Work in Progress (WIP) limits:** para mantener foco y evitar multitarea excesiva.

9. Métricas útiles vinculadas a estimación y planificación

- **Velocity (promedio y desviación estándar)** — para forecast.
- **Sprint burndown / burnup** — seguimiento diario del progreso.
- **Plazo de entrega estimado vs real** — medir precisión de estimaciones.
- **% de historias completadas vs comprometidas** — indicador de compromiso y planificación.
- **Lead time / Cycle time** — para entender flujo y cuellos de botella.

Consejo científico: usa **media + desviación** (ej.: $\text{media} \pm 1\sigma$) para presentar rangos de confianza, no un solo número. Esto comunica incertidumbre informada.

10. Errores comunes (y cómo evitarlos)

1. **Convertir puntos a horas de forma rígida:** evita prometer horas fijas; explica que la relación es empírica y específica del equipo.
2. **Mix de tareas técnicas y user stories sin separación:** prioriza distinguir deuda técnica y trabajo de entrega.
3. **Hacer estimaciones con stakeholders fuera del equipo:** solo el equipo de desarrollo estima; PO y stakeholders aportan contexto.
4. **Descomponer tarde:** historias demasiado grandes (epics) deben romperse antes del planning.
5. **Ignorar capacidad real** (reuniones, formaciones, permisos) — siempre descontar.
6. **No revisar estimaciones:** recalibrar cada 2–3 sprints.

11. Plantillas y checklists (listas prácticas)

Checklist para Sprint Planning

- PO presenta backlog priorizado y Sprint Goal propuesto.
- Equipo confirma capacidad neta (horas o puntos).
- Historias seleccionadas cumplen DoD mínimo para estimación.
- Historias grandes convertidas en tasks y tasks estimadas en horas.
- Riesgos y dependencias documentadas.
- Sprint Backlog cargado en herramienta (Jira/Trello/GitLab).
- Sprint Goal redactado y aceptado por equipo.

12. Actividades prácticas para aula/taller (dos opciones)

Actividad A — Estimación con Planning Poker (90 minutos)

Objetivo: estimar 20 historias de backlog.

Pasos:

1. Brief del caso (cooperativa cafetalera) y lectura del backlog (10 min).
2. Definir benchmark story (ej.: historia de 3 puntos) (5 min).
3. Sesión de Planning Poker en grupos (60–70 min).
4. Registrar estimaciones y discutir discrepancias (10–15 min).

Entregable: backlog con estimaciones y nota de decisiones.

Rúbrica (total 12 pts):

- Consistencia en aplicación de INVEST (0–4).
- Uso correcto del protocolo de Planning Poker (0–4).
- Justificación de estimaciones divergentes (0–4).

Actividad B — Proyección de Release y análisis de sensibilidad (120 minutos)

Objetivo: proyectar número de sprints para backlog de 200 puntos bajo 3 escenarios: optimista, base, conservador.

Pasos:

1. Calcular velocidad histórica o asumir (por ejemplo 30 base, 36 optimista, 24 conservador).
2. Calcular sprints necesarios para cada escenario (hacer divisiones paso a paso).
3. Añadir buffer (20%) y calcular tiempo calendario.
4. Presentar recomendaciones y argumentos.

Entregable: hoja de cálculo con cálculos y conclusiones.

13. Ejemplo aplicado completo (sprint planning realista para UPN — sistema de gestión estudiantil)

Contexto resumido: backlog inicial (MVP) 120 puntos. Equipo: 6 miembros (4 devs + 1 QA + 1 full-stack), velocidad histórica promedio = 24 puntos/sprint (por cargas administrativas y clases). Sprint length = 2 sem = 10 días.

¿Cuántos sprints para MVP?

1. sprints = $120 / 24$.
 - $24 \times 5 = 120 \rightarrow$ por tanto $120 / 24 = 5$.

Resultado: 5 sprints.

Capacidad real del sprint (ejemplo detallado)

- Horas brutas por miembro = $10 \times 8 = 80$ h.
- Horas totales brutas = $6 \times 80 = 480$ h.
- Factor no productivo (30%) $\rightarrow$ productivas = $480 \times (1 - 0.30) = 480 \times 0.70$.
 - $480 \times 0.70 = 336$ h.
- Horas por punto = $336 \text{ h} / 24 \text{ puntos} = 336 / 24$.
 - $24 \times 14 = 336 \rightarrow$ por tanto $336 / 24 = 14$ h/punto.

Interpretación: 1 punto $\approx$ 14 h en este equipo. Para una historia de 3 puntos $\rightarrow 3 \times 14 = 42 \text{ h} \approx 5.25$ días.

Buffer y tiempo calendario

- Añadir buffer 15%: sprints con buffer = $5 \times 1.15 = 5.75 \rightarrow$ redondear a 6 sprints = 12 semanas $\approx$ 3 meses.

Resultado: plan conservador = 6 sprints ($\sim$ 3 meses).

Implementación

En metodologías tradicionales, implementación se concebía como una fase final. En Scrum, la **implementación** es continua y ocurre en cada sprint:

- El equipo desarrolla incrementos de software **potencialmente entregables**.
- Cada incremento debe cumplir con el **Definition of Done** acordado.
- Se integran pruebas, revisiones y retroalimentación en ciclos cortos.
- El foco no es *terminar tareas* sino entregar valor verificable al usuario.

Clave: implementación = *transformar el backlog en incrementos de producto funcionando*, bajo un proceso empírico (inspección y adaptación).

2. Elementos críticos de la implementación

1. Entorno de desarrollo colaborativo

- Repositorio centralizado (GitHub/GitLab/Bitbucket).
- Estrategia de branching (ej.: GitFlow simplificado o trunk-based).
- Pipeline de CI/CD con pasos mínimos: build + tests + análisis estático.

2. Cadencia Scrum (implementación como práctica diaria)

- **Daily Scrum:** sincronización del equipo, detección temprana de impedimentos.
- **Sprint Planning:** selecciona ítems del backlog y define Sprint Goal.
- **Sprint Execution:** codificación, pruebas, integración continua.
- **Sprint Review:** demo de incrementos al PO y stakeholders.
- **Retrospective:** identificación de mejoras al proceso de implementación.

3. Prácticas de ingeniería recomendadas

- Desarrollo guiado por pruebas (**TDD**) o por comportamiento (**BDD**).

- Code reviews sistemáticas con checklist de calidad.
- Integración continua (CI) + despliegue continuo (CD) donde sea factible.
- Monitoreo automático de métricas (coverage, defect density, performance).
- Gestión de deuda técnica (backlog técnico y refactorización periódica).

4. **Monitoreo del progreso**

- Burn-down chart diario.
- Burn-up chart para medir valor entregado.
- Dashboard de calidad (test results, builds, coverage).

3. **Flujo operativo de implementación (paso a paso)**

1. **Seleccionar historias en Sprint Planning** → 8–10 historias con criterios claros.
2. **Descomponer historias en tasks** → tareas técnicas concretas (p. ej., “crear endpoint”, “escribir test unitario”).
3. **Asignar y coordinar trabajo** → equipo autoorganizado, tareas visibles en tablero (To Do / In Progress / Done).
4. **Desarrollo iterativo:**
 - Crear branch → implementar → escribir pruebas → pasar tests → hacer commit.
 - Pipeline CI ejecuta build y tests automáticamente.
 - Code review antes de merge a rama principal.
5. **Integración:** merge a rama principal (main/develop).
6. **Validación:** pruebas de aceptación y demo en Sprint Review.
7. **Entrega:** despliegue a staging o producción (según acuerdo).
8. **Feedback y mejora:** backlog actualizado con hallazgos.

4. Ejemplo numérico — seguimiento con Burn-down Chart

Contexto: Sprint de 2 semanas, capacidad planeada = 30 puntos.

- Día 0 (inicio): backlog sprint = 30 puntos.
- Día 5 (mitad de sprint): puntos completados = 12.
 - Puntos restantes = $30 - 12 = 18$.
- Línea ideal ($30 \text{ puntos} / 10 \text{ días} = 3 \text{ puntos/día}$).
 - Día 5 $\rightarrow 30 - (3 \times 5) = 30 - 15 = 15 \text{ puntos}$.
- Comparación: 18 puntos pendientes vs 15 esperados $\rightarrow 3 \text{ puntos por encima}$ (rezago leve).

Interpretación: equipo debe analizar impedimentos (quizá una historia sobredimensionada). Retrospective debe capturar esta lección.

5. Ejemplo aplicado — Sistema universitario UPN

Historia seleccionada:

“Como docente quiero subir las notas parciales al sistema para que los estudiantes las consulten en línea.”

Criterios de aceptación:

- Validar que cada nota esté entre 0 y 100.
- El sistema almacena fecha y autor del registro.
- Los estudiantes ven la nota en su perfil en $\leq 10 \text{ s}$ tras el registro.

Implementación paso a paso:

1. Crear branch feature/notas-parciales.
2. Escribir pruebas unitarias (nota válida/inválida, acceso restringido).
3. Codificar endpoint POST /notas.
4. Ejecutar pipeline: build OK, 95% coverage.
5. Code review: aprobado por 1 par.
6. Merge a rama principal y despliegue a staging.

7. Sprint Review: demo con 3 estudiantes → validación positiva.
8. Retro: agregar test de performance por carga masiva (200 estudiantes simultáneos).

6. Plantillas útiles para implementación

a) Checklist de Definition of Done (DoD)

- Código compilado sin errores.
- Tests unitarios ejecutados y pasan (coverage $\geq 70\%$).
- Code review aprobado.
- Documentación mínima actualizada.
- Pipeline CI pasa en todas las etapas.
- Historia validada en demo con PO.

b) Checklist de code review

- Legibilidad del código.
- Nombres claros y consistentes.
- Pruebas cubren casos normales y bordes.
- No se introducen vulnerabilidades conocidas.
- Documentación actualizada.

7. Riesgos frecuentes en la implementación y mitigaciones

<i>Riesgo</i>	<i>Ejemplo en Nicaragua</i>	<i>Mitigación</i>
<i>Baja conectividad interrumpe CI/CD</i>	Cooperativa rural con Internet inestable	Pipeline offline-first + sincronización nocturna
<i>Ausencia del PO para validar historias</i>	PO ocupado en labores administrativas universitarias	Establecer PO proxy o reuniones fijas de revisión
<i>Deuda técnica acumulada</i>	Código duplicado no refactorizado	Sprint con 15% capacidad para deuda técnica
<i>Pruebas insuficientes</i>	Entregas rápidas sin automatización	Incluir tests como criterio obligatorio de DoD

8. Actividad práctica para aula/taller — “Simulación de implementación”

Duración: 2–3 horas.

Objetivo: ejecutar el ciclo completo de implementación para una historia simple.

Pasos:

1. Cada grupo selecciona 1 historia (ej.: registro de estudiante, registro de saco de café).
2. Redactan criterios de aceptación (15 min).
3. Crean tareas técnicas y asignan (20 min).
4. Simulan commits en tablero (papel o Trello) con pipeline de validación.
5. Realizan “demo” a otros grupos (15 min).
6. Retro: identifican errores y lecciones (20 min).

Entregable: Sprint Backlog simulado + acta de demo con feedback.

Rúbrica (12 pts):

- Cumplimiento de DoD (0–4).
- Claridad de criterios de aceptación (0–4).
- Presentación de demo y registro de feedback (0–4).

9. Recomendaciones finales y consideraciones locales

1. **Arranca pequeño:** inicia pipelines simples (build + tests), luego expándelos.
2. **Cultura de calidad:** integrar DoD en la mentalidad del equipo, no solo en el papel.
3. **Monitoreo sencillo:** basta un burn-down en Excel para iniciar; herramientas avanzadas pueden venir después.
4. **Contexto nicaragüense:**
 - Diseñar soluciones *offline-first* para conectividad rural.
 - Documentar bien procesos para mitigar rotación de personal.
 - Involucrar a usuarios finales (estudiantes, cooperativistas) en demos; aumenta adopción.

Implementar en Scrum no es “programar hasta que duela”, sino *entregar valor en ciclos breves, con calidad, visibilidad y aprendizaje continuo*. Cada commit, cada prueba y cada demo es un paso firme hacia productos que resuelven problemas reales de Nicaragua.

Revisión y retrospectiva

- **Sprint Review (Revisión):** evento orientado a **inspección del Increment** y recolección de feedback de stakeholders con foco en valor y decisiones de negocio. Pregunta clave: *¿qué hemos logrado y qué debemos cambiar en el backlog?*
 - Resultado principal: producto inspeccionado + decisiones sobre el Product Backlog (re-priorización, ajustes).
- **Sprint Retrospective (Retrospectiva):** evento orientado a **inspección del proceso** y generación de acciones para mejorar cómo el equipo trabaja (calidad, procesos, herramientas, relaciones). Pregunta clave: *¿qué hicimos bien, qué no, y qué vamos a cambiar?*
 - Resultado principal: plan de mejora del proceso (lista priorizada de acciones con dueños).

Ambos eventos son complementarios y esenciales para que el empirismo (inspección → adaptación) funcione. Sin retrospectivas, la revisión se queda en mostrar resultados; sin review, la retro puede perder foco en el impacto del producto.

Reglas generales y buenas prácticas

1. **Timebox estricto:** respeta los límites (Review: 60 min máximo para 2-semanas; Retro: 45–90 min según equipo).
2. **Invitar a stakeholders relevantes a la Review** (clientes, usuarios, sponsor), pero mantener la Retro **sólo del equipo** (PO, SM, Dev Team; opcionalmente observador).
3. **Preparación previa:** demos ensayadas, criterios de aceptación validados; para la retro, reunir datos (burn-down, defects, MTTR, métricas de calidad).
4. **Evidencia y trazabilidad:** documenta acuerdos, actas, decisiones y acciones en el backlog/issue tracker (vínculo PR / commits si procede).

5. **Seguridad psicológica:** aplica la *Prime Directive* de Norm Kerth: “Asumimos que todos hicieron lo mejor que pudieron con el conocimiento, recursos y motivación que tenían” — evita culpas.
6. **Cierre con compromiso:** cada acción debe tener dueño y criterio de éxito (Definition of Done para la acción).

Sprint Review — guía práctica paso a paso

Propósito

Inspeccionar el Increment y adaptar el Product Backlog en base a feedback real de usuarios/stakeholders.

Asistentes recomendados

- Equipo de Desarrollo (todos)
- Product Owner
- Scrum Master (facilita logística)
- Stakeholders: clientes, usuarios, sponsor, representantes legales/contables si aplica

Timebox

- Sprint de 2 semanas: **60 minutos** (ajustar proporcionalmente para sprints más largos).

Agenda modelo (60 min)

1. **Apertura (5 min):** PO presenta objetivo del sprint y resumen del Sprint Goal.
2. **Demo del Increment (20–30 min):** el equipo muestra funcionalidades terminadas (centrarse en valor, no en código).
3. **Feedback estructurado (10–15 min):** stakeholders hacen preguntas y proponen cambios; registrar todo.
4. **Discusión de backlog y prioridades (5–10 min):** PO explica implicaciones y si propone re-priorizar.

5. **Cierre (2–5 min):** acuerdos, próximos pasos y responsable para registrar decisiones.

Plantilla de acta de Sprint Review (copiar + completar)

- Proyecto:
- Sprint N° / Fechas:
- Sprint Goal:
- Asistentes (roles):
- Incrementes presentados (IDs / historias):
- Feedback clave de stakeholders (listado):
- Decisiones sobre backlog (añadidos / repriorizados / bloqueados):
- Riesgos emergentes identificados:
- Próximos pasos y responsables:
- Firma/confirmación (PO): fecha.

Buenas prácticas para la demo

- Hacer la demo en un entorno estable (staging) o grabada si conectividad es un problema.
- Mostrar **casos de uso** reales (por ejemplo, registrar 3 entradas de sacos de café con fotos) en vez de pantallas sin contexto.
- Usar datos representativos (anonimizados si es necesario) para validar workflows.

Ejemplo (mini)

En la cooperativa “Cosecha Viva”, demo: registrar ingreso de saco → se muestra QR y sincronización asíncrona. Stakeholder (tesorero) solicita reportes por lote por fecha → PO lo marca como alta prioridad y crea historia US-XX en backlog.

Retrospective — guía práctica detallada

Propósito

Mejorar el proceso: identificar qué funcionó, qué no y crear acciones concretas (inspección→adaptación del proceso).

Asistentes

- Dev Team (todos)
- Scrum Master (facilitador)
- Product Owner (opcional; preferible que asista y participe en diálogo si el equipo lo acepta)

Nota: si la presencia del PO inhibe la conversación candid, considerar la retro sin PO o con PO en una parte limitada (por ejemplo, escucha de acciones pactadas).

Timebox

- Sprint de 2 semanas: **60 minutos** recomendado; mínimo 45, máximo 90 según tamaño y complejidad.

Preparación previa del facilitador

- Reunir datos: burndown, defectos, MTTR, métricas de calidad, % cumplimiento DoD, feedback de Review.
- Elegir formato/metodología (Start-Stop-Continue, 4Ls, Sailboat, Mad/Sad/Glad, Lean Coffee).
- Preparar material: tablero, notas adhesivas, herramienta remota (Miro, Mural, Jamboard) o física.

Estructura general (60 min ejemplo)

1. **Apertura & Prime Directive (5 min):** recordatorio de reglas (respeto, no culpas).
2. **Datos y contexto (5–10 min):** presentador (SM/PO) muestra métricas clave.
3. **Generar observaciones (15–20 min):** brainstorming silencioso -> sticky notes (qué fue bien, qué no, ideas).

4. **Cluster & Insight (10–15 min):** agrupar notas, discutir temas principales, hacer 5 Whys si procede para raíz.
5. **Priorizar acciones (10–15 min):** dot-voting para elegir top 3 acciones; escribir dueño y criterio de éxito.
6. **Cierre (5 min):** ROTI (Return On Time Invested) rápido y confirmación de dueños.

Técnicas y formatos (elige según necesidad)

- **Start / Stop / Continue:** simple y efectivo.
- **4Ls (Liked / Learned / Lacked / Longed For):** promueve matices.
- **Sailboat:** identifica fuerzas que impulsan y bloqueos (anclas) hacia la meta. Muy útil con stakeholders externos en mente.
- **Lean Coffee:** agenda democrática para temas emergentes.
- **5 Whys + Fishbone:** para análisis de causa raíz cuando hay problemas repetidos.
- **Appreciative Inquiry:** focaliza fortalezas y cómo potenciarlas.

Cómo convertir hallazgos en acciones efectivas

- Para cada acción, definir: **Descripción, Dueño, Criterio de éxito (DoD acción), Tiempo de verificación** (ej.: en next retro), **Esfuerzo estimado** (p. ej., spike 1 día).
- Registrar la acción en el backlog (como tarea técnica o ticket) y **no** dejarla sólo en el documento de la retro.
- Preferir acciones pequeñas y medibles (SMART): ej., "Automatizar test de endpoint de sincronización con cobertura mínima 80%" > en vez de "mejorar pruebas".

Plantilla de registro de acciones (Action Register)

- Acción ID | Descripción | Dueño | Prioridad | Criterio de Éxito | Fecha inicio | Fecha revisión | Estado

Ejemplo de retro (cooperativa — resultado)

- Observación: sincronización falla con fotos grandes.
- Acción A1: implementar compresión y retries (Dueño: DevLead; Criterio: sin fallos > 90% en 2 semanas).
- Observación: operadores no entienden validaciones de peso.
- Acción A2: mejorar mensajes de error y breve guía impresa (Dueño: PO; Criterio: 80% adopción en 1 mes).
- Observación: dailies demasiado largas.
- Acción A3: limitar updates a 2 puntos por persona y usar parcheo asíncrono (Dueño: SM; Criterio: daily $\leq$ 15 min por 3 sprints).

Medición de efectividad — métricas para Review y Retro

- **% acciones de la retro implementadas al siguiente sprint** (ideal $\geq$ 70%).
- **ROTI promedio** (Return On Time Invested) por retro (1–5 escala; meta $\geq$ 4).
- **Satisfacción del equipo con la retro** (post-retro poll 1–5).
- **Tasa de incorporación de feedback de Review al backlog** (nº items creados / nº sugerencias).
- **Lead time de acciones de mejora** (días desde acción registrada hasta verificación).
- **Reducción de incidentes repetidos** (p. ej., decrease in escaped defects % tras mitigación).

Registra estas métricas en una hoja de seguimiento y revísalas trimestralmente para verificar que las retros están generando impacto.

Facilitación avanzada — pautas del facilitador (SM)

1. **Preparar datos y mood:** revisa métricas y tono del equipo; si la moral está baja, iniciar con ejercicio de reconocimiento (Mad/Sad/Glad).
2. **Silencio productivo:** usar brainstorming silencioso para evitar dominación por voz de figuras fuertes.
3. **Dot-voting estructurado:** cada persona tiene N puntos para priorizar; evita votaciones populistas.
4. **Trabajar con hechos:** evita juicios; pide evidencias (logs, builds, tickets).
5. **Control de tiempo estricto:** usa temporizador y transiciones claras.
6. **Registrar y rastrear:** crea tickets inmediatamente y asigna responsables; mostrar en next retro si la acción progresó.
7. **Manejar conflictos:** usar técnicas de mediación breve; si hay conflicto larvado, proponer sesión aparte de resolución.
8. **Fomentar experimentos:** tratar acciones como hipótesis (minimizar dogma): “Probaremos X por 2 sprints y mediremos Y”.

Manejo en contextos con baja conectividad o recursos limitados

- **Review remoto con conectividad limitada:** grabar demo en dispositivo local y llevar archivo físico / USB a stakeholders o enviar por WhatsApp (compresión).
- **Retro presencial con recursos limitados:** usar papel y pizarras; fotografiar resultados y subir cuando haya conexión.
- **Stakeholders remotos:** enviar resumen ejecutivo (1 página) por correo/WhatsApp con decisiones en vez de exigir presencia sin garantías de conectividad.

Errores comunes y cómo evitarlos

1. **Dejar acciones en la retro sin tickets** → riesgo: no se implementan. *Solución:* crear ticket inmediatamente y asignar dueño.
2. **Retros repetitivas y sin novedad** → aburrimiento. *Solución:* cambiar formato, traer datos nuevos, limitar a acciones pequeñas y medibles.
3. **Review sin stakeholders relevantes** → feedback pobre. *Solución:* invitar stakeholders clave y negociar asistencia mínima o enviar demo grabado.
4. **Culpar individuos en la retro** → deterioro del equipo. *Solución:* aplicar Prime Directive y enfocarse en sistemas/procesos.
5. **No medir efectividad** → no sabrás si la retro aporta valor. *Solución:* medir ROTI, % acciones completadas y satisfacción.

Actividad práctica (taller) — “Revisión + Retrospectiva completa” (simulación)

Duración sugerida: 2 horas (Review 45–60 min + Retro 60 min, o dividir en dos sesiones).

Materiales: backlog, incremento (demo), burn-down chart, impresos o pizarra, notas adhesivas.

Pasos:

1. **Sprint Review (45–60 min):** demo + feedback + acta (PO registra decisiones).
2. **Recopilación de datos (5 min):** SM prepara métricas clave.
3. **Retrospective (60 min)** usando formato **Sailboat** o **Start-Stop-Continue**.
4. **Creación de actions:** dot-voting y asignación de dueños.
5. **Registro:** crear tickets y calendarizar verificación en próxima retro.
6. **Cierre y ROTI.**

Entregable de cada grupo: acta de Review + Action Register con ≥ 3 acciones con dueños y criterios.

Rúbrica de evaluación (12 pts):

- Calidad de la demo y claridad del feedback (0–4)
- Relevancia y factibilidad de las acciones de la retro (0–4)
- Registro y trazabilidad (tickets creados, dueños asignados) (0–4)

Plantillas rápidas para copiar/pegar

Acta de Sprint Review (texto)

Proyecto: _____

Sprint #: ____ Fechas: ____ - ____

Sprint Goal: _____

Incrementos presentados:

- US-__ : [título] — estado: Done / Not Done

Feedback clave de stakeholders:

1. _____

2. _____

Decisiones tomadas:

- Priorizar US-__ para el próximo sprint. (PO)

- Crear spike para evaluar X. (Dueño: __)

Riesgos detectados:

- _____

Próximos pasos:

- Responsable: _____ Fecha: _____

Registro de acciones (Retrospective)

ID | Acción | Dueño | Prioridad | Criterio de éxito (DoD) | Fecha inicio | Fecha revisión | Estado

A1 | Comprimir fotos en cliente | DevLead | Alta | <1% fallos en sync en 2 wks | 2025-09-10 | 2025-09-24 | Abierto

Cómo integrar outputs de Review/Retro en el flujo del backlog

1. **Feedback de Review** → PO crea historias/epics en Product Backlog y prioriza (marcar como "from-review" para trazabilidad).
2. **Acciones de Retro** → crear tickets en backlog técnico o sprint backlog según prioridad; preferir tickets pequeños y con evidencia.
3. **Verificación** → en la siguiente Retrospective verificar el estado y medir impacto según criterio de éxito.
4. **Comunicación** → informar a stakeholders (via acta) de acciones que afectan funcionalidad o cronograma.

Revisión y retrospectiva no son ceremonias administrativas: son el **motor del aprendizaje** en Scrum. Ejecutadas con disciplina, datos y respeto generan un ciclo de mejora real y sostenido. En contextos nicaragüenses, poner énfasis en la demostración con datos de campo (ej.: fotos, registros) y en acciones pequeñas y verificables acelera la adopción y mantiene la confianza de stakeholders locales.

Lanzamiento

En Scrum, el **lanzamiento (release)** no es un evento único sino la culminación de ciclos iterativos: un conjunto de increments que alcanzan la Definition of Done y los criterios de aceptación del negocio. El lanzamiento es el momento en que el producto (o un incremento significativo) pasa al uso real por usuarios finales o stakeholders. Se trata de **mitigar riesgos, asegurar calidad, maximizar adopción y minimizar impacto** en operación.

El lanzamiento debe ser tratado como un experimento empírico: planificar, ejecutar, medir y adaptar. Un lanzamiento mal gestionado puede erosionar confianza, generar costos de corrección altos y frenar adopción —especialmente crítico en contextos con recursos limitados como muchas PYMEs y cooperativas en Nicaragua.

Tipos de estrategia de lanzamiento

1. Big-Bang / Full Release

- *Descripción:* liberar todo el producto a todos los usuarios en una única fecha.
- *Ventajas:* simple de coordinar, clara ruptura de versión.
- *Desventajas:* alto riesgo y coste de rollback; no aconsejable para sistemas críticos o donde el cambio es grande.
- *Uso:* puede usarse para MVP in-house o entornos de laboratorio, pero evitar con usuarios críticos.

2. Pilot → Rollout (Recomendado para contextos nicaragüenses)

- *Descripción:* desplegar primero a un grupo control (p. ej., una sede o cooperativa piloto), validar métricas y luego escalar.
- *Ventajas:* baja exposición inicial, feedback real, posibilidad de corregir antes de escala.

- *Uso:* ideal para cooperativas rurales y universidades.

3. **Canary Release**

- *Descripción:* desplegar a un pequeño % de usuarios en producción y aumentar progresivamente si métricas son positivas.
- *Ventajas:* detecta problemas tempranos en producción; permite monitoreo real con menor blast radius.
- *Requisitos:* infraestructura capaz (feature flags, segmentación de tráfico).

4. **Blue-Green / Rolling**

- *Descripción:* mantener dos entornos (blue = current, green = new); switch de tráfico al green cuando se valide; o despliegues rolling por instancias.
- *Ventajas:* rollback rápido, menor downtime.
- *Requisitos:* capacidad para duplicar entorno de producción (costo).

5. **Feature Toggles (Feature Flags)**

- *Descripción:* habilitar funcionalidad en producción via flag; permite activar/desactivar sin despliegue.
- *Ventajas:* control fino por usuario/grupo; muy útil para pruebas A/B y canaries.
- *Caveat:* requiere disciplina y limpieza posterior (remover flags).

Criterio práctico para elegir estrategia: tamaño del cambio, criticidad del sistema, capacidad técnica de infraestructura, capacidad de soporte y sensibilidad regulatoria. Para proyectos en Nicaragua (cooperativas, UPN), **Pilot** → **Rollout** combinado con feature flags y canaries suele ser la opción más prudente.

Release Plan — estructura mínima (plantilla explicada)

Título del Release:

Versión / Tag (SemVer): e.g., v1.2.0 (usar SemVer: MAJOR.MINOR.PATCH).

Fecha objetivo de Go-Live:

Alcance (features incluidas): — lista de historias/epics (IDs).

Stakeholders / Sponsor: — lista y canales.

Estrategia de despliegue: (Pilot → Rollout / Canary / Blue-Green etc.)

Ambientes: (Dev → QA → Staging → Production).

Criterios de release (pre-go): (lista de checks que deben pasar).

Plan de rollback: (cómo revertir, tiempo objetivo).

Plan de comunicaciones: (quién notifica a quién y cuándo).

Plan de soporte post-lanzamiento: (on-call rota, SLAs).

KPIs y umbrales de alarma: (uptime, error rate, adoption rate, MTTR).

Riesgos y mitigaciones: (top 5).

Checklist de go/no-go (sí/no por item).

Aprobaciones (firmas PO / Sponsor / SM):

Criterios de release (Go/No-Go) — checklist ampliado

Antes de habilitar el switch a producción, TODOS los criterios marcados como *mandatorios* deben ser "PASSED". Ejemplo:

Criterios técnicos

- Todos los tests automatizados (unit + integration) en pipeline **pass**.
- Quality gate (SonarQube): No **blocker/critical** issues.
- Build estable en main y artefacto versionado y signado.
- Smoke tests en *staging* pasan (automáticos o manuales).
- Backup de DB tomado y verificado (backup + checksum).
- Scripts de migración probados en *staging* y reversibles.

Criterios operativos

- Plan de despliegue documentado y probado (DRY-run).

- Plan de rollback testado (simulación ok).
- On-call y soporte confirmados (nombres, contactos, horario).
- Entrenamiento mínimo dado a usuarios clave (PO/operadores) y manuales disponibles.
- Licencias/contratos y cumplimiento regulatorio confirmados (si aplica).

Criterios de negocio

- Product Owner y Sponsor aprueban la versión (firma digital o correo).
- Stakeholders clave informados y fecha aceptada.
- KPIs objetivo definidos y dashboard listo.
- Plan de comunicación público (release notes) preparado.

Plan de rollback (plantilla y ejemplos)

Rol crítico: el plan de rollback debe ser claro, probado y con responsables definidos. Contenido mínimo:

- **Trigger(s) para rollback:** condiciones concretas que activan rollback (p. ej., error de >2% en transacciones críticas durante 30 min, fallo de migración que deja registros inconsistentes).
- **Rollback steps (paso a paso):**
 1. Notificar equipo y stakeholders.
 2. Activar modo mantenimiento (si aplica).
 3. Revertir switch (blue→green) o desactivar feature flag.
 4. Restaurar base de datos desde backup (si necesario) con procedimiento y checksum.
 5. Validar integridad (sanity checks).
 6. Informar resolución y plan de acciones post-mortem.
- **Tiempos objetivo:** RTO (Recovery Time Objective), RPO (Recovery Point Objective). Ej.: $RTO \leq 2 \text{ h}$; $RPO \leq 15 \text{ min}$.
- **Responsables:** Nombre, contacto, rol para cada paso.

- **Pruebas previas:** resultados de simulaciones (fecha y observaciones).

Nota: para bases de datos con cambios no triviales (migraciones destructivas), preferir **backwards-compatible migrations** o estrategia de dual-write + back-fill en vez de revert absoluto, porque la restauración puede implicar pérdida de transacciones de usuarios.

Versionamiento y artefactos de release

- **SemVer recomendado:** MAJOR.MINOR.PATCH
 - MAJOR: cambios incompatibles API.
 - MINOR: nuevas funcionalidades compatibles.
 - PATCH: corrección de bugs.
- **Tagging en VCS:** `git tag -a v1.2.0 -m "Release v1.2.0"` y establecer artefactos asociados (docker image tag, package).
- **Artefactos a generar y publicar en release:** paquete binario/imagen, checksum, release notes, migration scripts, runbooks, lista de endpoints/API changes, credentials/keys (si aplica) rotadas, y manifiesto de dependencias (licenses).

Notas de lanzamiento (Release Notes) — plantilla y ejemplo corto

Plantilla

- Versión: vX.Y.Z
- Fecha: YYYY-MM-DD
- Resumen ejecutivo (1–2 frases)
- Nuevas funcionalidades (lista breve)
- Correcciones y mejoras (lista)
- Cambios de compatibilidad / migraciones (impacto y acciones requeridas)
- Requisitos previos (p. ej., mantenimiento, downtime)
- Cómo reportar problemas (contacto on-call, formulario)
- Enlaces: User Guide, API Docs, Runbook.

Ejemplo (cooperativa — release piloto)

Versión: v1.0.0-pilot

Fecha: 2025-09-15

Resumen: Primera liberación piloto del módulo de registro de sacos para la cooperativa “Cosecha Viva”.

Nuevas funcionalidades:

- Registro en campo con foto y QR (offline sync).
- Generación de reporte por lote en CSV.

Correcciones:

- Validación de peso corregida para rangos locales.

Impacto:

- Se requiere sincronización inicial en oficina central (3–5 min). No se requiere downtime.

Reportar incidencias: soporte@cooperativa.org — on-call: +505-XXX (08:00–18:00)

Comunicación y coordinación (plan de comunicaciones)

Antes del lanzamiento

- Notificación a stakeholders de la ventana de despliegue (fecha/hora), impactos esperados y contactos.
- Envío de material de capacitación breve (1-page cheat sheet) a usuarios claves.
- Revisión de plan de rollback y confirmación de on-call.

Durante la ventana

- Canal de comunicación dedicado (Slack/WhatsApp + conferencia telefónica).
- Status messages cada 15–30 minutos (start, progress, validation, success/failure).
- Registro de eventos (log de acciones) por persona responsable.

Después del lanzamiento

- Comunicado de éxito o incidente con resumen y plan de acción.
- Envío de release notes a usuarios.
- Primera verificación de adopción y métricas a 24–72 h.

Monitoreo post-lanzamiento y KPIs críticos

KPIs técnicos

- Uptime / Availability (objetivo 99.x% según criticidad).
- Error rate (5xx per 1000 requests).
- Latency (p95, p99) en endpoints críticos.
- MTTR (Mean Time To Recover) para incidentes críticos.
- Number of critical incidents in first 72h.

KPIs de negocio / adopción

- Adoption rate (usuarios activos / total target) en 7/30 días.
- Task completion rate (ej., % de registros completados sin error).
- Support tickets by severity (count / day).

- Satisfaction score (survey 1–5) tras 7 días.

Thresholds y alertas

- Definir umbrales por KPI con acciones asociadas (ej., si error rate > 2% durante 30 min → activar comité de emergencia y considerar rollback).

Soporte y operación post-lanzamiento

On-call rota (primeras 72 horas crítica): asignar equipo con horarios y escalamiento claro (1st, 2nd, 3rd level).

Runbook/Playbooks: procedimientos para incidentes comunes (login fails, DB lock, failed migrations).

Ticketing y SLAs: registrar tickets con prioridad y tiempos de respuesta (ej., P1 response ≤ 30 min, P1 resolution ≤ 4 h).

Capacity planning: prever recursos en hosting para picos iniciales (ej., release de matrícula en UPN durante apertura de inscripciones).

Consideraciones regulatorias y de cumplimiento (contexto Nicaragua)

- **Facturación electrónica (DGI):** si el release implica cambios en facturación, incluir validación con contador y pruebas de formato; coordinar con DGI si es necesario.
- **Protección de datos:** cumplir con normativa local (y buenas prácticas internacionales) en almacenamiento y transmisión; controles de acceso.
- **Contratos/entregables públicos:** en proyectos con financiación pública, verificar cláusulas de aceptación y documentación requerida para la recepción formal.
- **Certificaciones/inspecciones:** algunas aplicaciones (salud, educación) pueden requerir revisiones formales antes del uso operativo.

Prácticas técnicas clave para un lanzamiento seguro (checklist técnico)

- Infraestructura redundante para producción (si aplica).
- Backups y snapshots verificados y accesibles.
- Feature toggles configurables por entorno.
- Monitoreo y alertas configuradas y probadas.
- Logs centralizados y retención configurada.
- Pipeline de CI con paso de release y artefactos signados.
- Seguridad: rotación de credenciales, keys, y escaneo de vulnerabilidades.
- Tests de carga/metadatos de performance en staging.
- Documentación de despliegue y rollback (runbooks).

Riesgos típicos en el lanzamiento y mitigaciones

- **Riesgo:** errores en migración de datos → *Mitigación:* dry-run en staging, backup, plan de rollback.
- **Riesgo:** incapacidad de soporte (no hay personal durante ventana) → *Mitigación:* acordar on-call y escalamiento.
- **Riesgo:** baja aceptación por usuarios → *Mitigación:* piloto, training, material de ayuda.
- **Riesgo:** incumplimiento regulatorio → *Mitigación:* revisión legal y pruebas de cumplimiento previas.
- **Riesgo:** impacto por baja conectividad → *Mitigación:* usar sincronización asíncrona y fases de rollout locales.

Ejemplos aplicados

Ejemplo A — Cooperativa cafetalera (Pilot → Rollout)

1. **Plan:** Pilot en 1 bodega de Matagalpa (10 usuarios) por 2 semanas.
2. **Pre-go:** Tests unitarios/integ en staging, demo a PO/encargado, backup DB, on-call 24/7 primer fin de semana.
3. **Go:** desplegar pilot (feature flag ON para bodega X). Monitoreo 72 h continuo.
4. **Evaluación:** métricas adoption ($\geq 70\%$ uso diario), error rate $< 1\%$ → si OK, programar rollout por municipios cada 2 semanas.
5. **Rollback:** desactivar flag y evaluar issues.

Ejemplo B — Universidad Politécnica (lanzamiento módulo de matrícula)

- **Ventana de release:** fuera de horario de matrícula (por la noche) y con soporte en oficina por 48 h.
- **Pasos adicionales:** pruebas de carga simulando pico de matrícula; coordinación con área TI de la universidad; manuales y sesiones presenciales para secretaría académica.
- **KPIs:** tiempo medio de matrícula por estudiante, % transacciones exitosas, incidentes críticos por hora.

Ejemplo C — PYME de Managua (release menor)

- **Estrategia:** canary 10% de usuarios → 50% → 100% en horas/ días con monitoreo.
- **Rollback:** feature flag para desactivar rápidamente.
- **Comunicación:** email a clientes y cheat-sheet de 1 página.

Actividad práctica para aula/taller — “Simula un lanzamiento piloto”

Duración: 3 horas (planificación + ejecución simulada)

Objetivo: preparar y ejecutar un pilot release para una cooperación/cooperativa o módulo UPN.

Pasos:

1. **Crear Release Plan** (45 min): alcance, estrategia, criterios, rollback, comunicación.
2. **Completar checklist de go/no-go** (30 min).
3. **Simular despliegue** (45–60 min): roleplay con equipo: DevOps ejecuta steps, SM coordina comunicaciones, PO valida demo; simular fallo y ejecutar rollback.
4. **Analizar métricas y decidir** (30 min): con datos de simulación, decidir si se escala o se reprogama.
5. **Entregable:** Release Plan + registro de la simulación (logs, decisión).

Rúbrica (12 pts):

- Exhaustividad del Release Plan (0–4).
- Calidad del checklist y pruebas de rollback (0–4).
- Decisión justificada basada en métricas simuladas (0–4).

Post-lanzamiento: verificación, lecciones y release retrospective

- **Verificación inmediata (24–72 h):** comparar KPIs contra umbrales; confirmar adopción; cerrar incidentes P1.
- **Release Retrospective (1 semana después):** reunir equipo y stakeholders para revisar qué funcionó, qué no, y actualizar runbooks y backlog con acciones (ej., mejorar scripts de migración, aumentar cobertura de tests).
- **Documentar lecciones:** actualizar plantillas y checklist para siguientes releases.

Plantillas listas para copiar (texto)

1) Checklist Go/No-Go

- ☐ Tests unitarios: PASSED
- ☐ Tests integración: PASSED
- ☐ Smoke tests en staging: PASSED
- ☐ Backups verificados con checksum: YES
- ☐ Rollback plan probado: YES
- ☐ On-call definido con contactos
- ☐ Release notes y user guide listos
- ☐ Stakeholders notificados
- ☐ Approval PO/Sponsor: SIGNED

2) Release Notes breve (ejemplo ya mostrado arriba)

3) Plan de rollback (forma compacta)

Trigger:

- Error rate > X% durante Y minutos
- Data corruption detected on table Z

Rollback Steps:

1. Notify: channel + phone
2. Enable maintenance page
3. Switch traffic to Blue (or turn off feature flag)
4. Restore DB from snapshot YYYY-MM-DD-HHMM if needed
5. Verify sanity checks A,B,C
6. Inform stakeholders & open post-mortem

Métricas de éxito (cómo saber que el lanzamiento fue exitoso)

- **Técnico:** Error rate < threshold; MTTR dentro del objetivo; 0 incidentes críticos en 72 h (meta).
- **Adopción:** $\geq$ target adoption rate (ej., 60–80% usuarios objetivo en 7 días para pilot).
- **Satisfacción:** Survey promedio $\geq$ 4/5 a los usuarios clave tras 7 días.
- **Operativo:** Tickets P1 resueltos en SLA; release retrospective con $\geq$ 1 acción implementada antes del siguiente release.

Recomendaciones finales y buenas prácticas

1. **Lanza como ensayos científicos:** usa pilotos, define hipótesis (p. ej., “esperamos 70% adopción en 30 días”) y mide; si la hipótesis falla, adapta.
2. **No lances solo por cumplir calendario:** prioriza calidad y soporte. Mejor una liberación más tarde y estable que una apresurada y problemática.
3. **Prepara a la gente:** entrenamiento breve y cheat-sheets aumentan adopción y reducen tickets iniciales.
4. **Automatiza lo que puedas:** pipelines, feature flags y checks reducen riesgos humanos.
5. **Documenta todo:** un buen release notes + runbook es oro cuando algo falla.

6. **Piensa en el contexto local:** sincronización offline, ventanas de despliegue respetando horarios y feriados, y comunicación por canales populares (WhatsApp) cuando el email no es suficiente.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... Thomas, D. (2001). **Manifesto for Agile Software Development**. Agile Alliance.

Schwaber, K., & Sutherland, J. (2020). **The Scrum Guide** (2020 ed.). Scrum.org.

Sutherland, J. (2014). **Scrum: The Art of Doing Twice the Work in Half the Time**. Crown Business.

Cohn, M. (2005). **Agile Estimating and Planning**. Prentice Hall.

Rubin, K. S. (2012). **Essential Scrum: A Practical Guide to the Most Popular Agile Process**. Addison-Wesley.

Sommerville, I. (2011). **Ingeniería de software** (9.^a ed.). Pearson Educación.

Pressman, R. S., & Maxim, B. R. (2015). **Ingeniería de software: Un enfoque práctico** (8.^a ed.). McGraw-Hill Educación.

Project Management Institute. (2017). **A Guide to the Project Management Body of Knowledge (PMBOK® Guide) — Sixth Edition**. Project Management Institute.

Highsmith, J. (2002). **Agile Software Development Ecosystems**. Addison-Wesley.

Poppendieck, M., & Poppendieck, T. (2003). **Lean Software Development: An Agile Toolkit**. Addison-Wesley.

Kerth, N. (2001). **Project Retrospectives: A Handbook for Team Reviews**. Dorset House.

Dingsøyr, T., Nerur, S., Balijepally, V., & Moe, N. B. (2012). **A decade of agile methodologies: Towards explaining agile software development**. *Journal of Systems and Software*, 85, 1213–1221.

Adopción (de tecnología / proceso)

Proceso por el cual una organización incorpora una práctica, herramienta o marco (p. ej., Scrum) en su operación. Incluye formación, pilotaje, cambios en gobernanza y evidencia de uso. En contextos nicaragüenses suele requerir pilotos locales y capacitación de "champions".

Agile (Ágil)

Familia de marcos y prácticas para desarrollo de software basados en iteración, entrega incremental y empirismo (inspección, adaptación, transparencia).

Respaldado por el Manifiesto Ágil.

Affinity Estimation (Estimación por afinidad)

Técnica colaborativa para estimar grandes cantidades de ítems agrupándolos por similitud (S/M/L/XL) y asignando valores relativos, útil cuando se tiene mucho backlog.

Artifact (Artefacto)

En Scrum, objeto tangible de trabajo y control (p. ej., Product Backlog, Sprint Backlog, Incremento) que contiene información necesaria para la inspección y decisión.

Backlog (Product Backlog / Sprint Backlog)

Lista priorizada de requisitos/funcionalidades (Product Backlog) o de las tareas comprometidas para un sprint (Sprint Backlog). Herramienta central de priorización y planificación.

Balance de capacidad / Capacity Planning

Estimación de la capacidad real de un equipo en un sprint (horas productivas o puntos), descontando tiempo no productivo (reuniones, formación, ausencias).

Big-Bang Release

Estrategia de liberación que entrega todo el cambio a todos los usuarios simultáneamente; de alto riesgo y generalmente no recomendada para entornos críticos.

Blue-Green Deployment

Estrategia de despliegue que mantiene dos entornos (blue, green) y cambia el tráfico al nuevo entorno tras validar; facilita rollback rápido.

Burn-down Chart

Gráfico que muestra puntos (o esfuerzo) pendientes a lo largo del sprint; se usa para monitorizar el progreso diario y detectar desviaciones.

Burn-up Chart

Gráfico que representa el trabajo completado frente al total planificado, útil para visualizar alcance y progreso acumulado.

Canary Release

Despliegue progresivo de una versión a un pequeño porcentaje de usuarios para observar comportamiento en producción antes de escalar.

Criterios de aceptación

Conjuntos de condiciones que debe satisfacer una historia de usuario para ser considerada completada y aceptada por el Product Owner.

Daily Scrum (Daily)

Reunión diaria de máximo 15 minutos en la que el equipo sincroniza el trabajo, detecta impedimentos y ajusta el plan operativo del día.

Definition of Done (Definición de "Hecho")

Conjunto explícito de condiciones que debe satisfacer un incremento para considerarse entregable (tests, documentación, despliegue en staging, etc.).

Deuda técnica

Compromisos de calidad técnica (atajos, código duplicado, falta de tests) que, si no se abordan, incrementan coste y riesgo futuro; debe gestionarse y priorizarse.

Despliegue Canary

Ver Canary Release; verificación controlada y progresiva de versiones en producción.

DevOps

Conjunto de prácticas que integran desarrollo y operaciones para acelerar el ciclo de entrega mediante CI/CD, automatización y retroalimentación continua.

Epic

Elemento del backlog de gran tamaño que se divide en historias de usuario más pequeñas para planificar y ejecutar.

Escenario (BDD)

Descripción estructurada de comportamiento (Given–When–Then) usada en BDD para definir pruebas de aceptación legibles por negocio.

Escalamiento (de Scrum)

Prácticas o marcos (p. ej., Scrum of Scrums, Nexus, SAFe) para coordinar múltiples equipos que trabajan en el mismo producto o producto complejo.

Estimación relativa

Aproximación del esfuerzo basada en comparación entre ítems (p. ej., story points) en lugar de medidas absolutas de tiempo.

Expected Monetary Value (EMV)

Medida cuantitativa de riesgo: $EMV = \text{Probabilidad} \times \text{Impacto monetario}$. Utilizada para priorizar mitigaciones por exposición financiera.

Feature Toggle (Feature Flag)

Mecanismo técnico para activar/desactivar funcionalidades en producción sin desplegar, permitiendo pruebas y canaries controlados.

Fibonacci (serie usada en Planning Poker)

Secuencia adaptada (0,1,2,3,5,8,13,20...) utilizada en Planning Poker para reflejar incertidumbre creciente en estimaciones.

Incremento (Increment)

Suma de todos los Product Backlog items completados durante un sprint y anteriores, que debe estar potencialmente desplegable según la DoD.

Impedimento

Cualquier bloqueo que impide el progreso del equipo; el Scrum Master facilita su eliminación o escalamiento.

Integración continua (CI)

Práctica de integrar cambios frecuentemente en un repositorio compartido y ejecutar builds/tests automáticos para detectar regresiones temprano.

Iteración / Sprint

Periodo de tiempo fijo (típicamente 1–4 semanas) durante el cual se transforma parte del Product Backlog en un Incremento potencialmente entregable.

Kanban

Método visual para gestionar flujo de trabajo (tablero con columnas To Do / Doing / Done), que enfatiza límites de WIP y mejora del flujo continuo.

KPI (Key Performance Indicator)

Indicador clave de desempeño usado para medir objetivos (p. ej., MTTR, adoption rate, uptime, velocity).

Lean

Enfoque de gestión orientado a eliminar desperdicios y maximizar valor; aplicado al software con principios de flujo, calidad integrada y mejora continua.

Lecciones aprendidas / Retrospectiva

Registro y análisis de lo aprendido en un sprint o release, que debe traducirse en acciones concretas y verificables.

Licencia / Compliance

Aspectos legales y regulatorios (p. ej., facturación electrónica del DGI, protección de datos) que condicionan diseño, pruebas y despliegue.

Mínimo Producto Viable (MVP)

Versión mínima del producto que permite validar hipótesis de negocio con el menor esfuerzo posible (útil para pilotos en cooperativas/UPN).

Monitoring / Observabilidad

Conjunto de métricas, logs y trazas que permiten detectar incidencias, analizar rendimiento y tomar decisiones operativas post-lanzamiento.

MTTR (Mean Time To Recover/Repair)

Tiempo promedio requerido para recuperar el servicio tras una incidencia crítica; KPI clave de operaciones.

Negociable (N de INVEST)

Característica de las historias: deben poder discutirse y refinarse, no ser contratos fijos rígidos.

Obstáculo / Bloqueador

Ver impedimento — evento que detiene o reduce la capacidad del equipo.

Off-line first

Estrategia técnica que prioriza el funcionamiento sin conexión y sincronización diferida, crítica para zonas con baja conectividad (ej.: trabajo de campo en Matagalpa).

Persona (user persona)

Representación ficticia de un usuario objetivo que ayuda a priorizar funcionalidades y diseñar criterios de aceptación.

Planning Poker

Técnica de estimación en la que cada miembro del equipo vota en secreto una carta con puntos; promueve consenso y evita anclaje.

Portafolio

Conjunto de proyectos/productos gestionados con criterios de priorización y financiación a nivel organizacional.

Product Backlog

Listado dinámico y priorizado de requisitos que mantiene el Product Owner; fuente única de trabajo para el equipo.

Product Owner (PO)

Rol responsable de maximizar el valor del producto: prioriza el backlog, define criterios de aceptación y representa a stakeholders.

Quality Gate

Condición técnica (por ejemplo: cobertura mínima, sin vulnerabilidades críticas) que debe cumplirse para permitir merge o despliegue.

Radar de riesgos / Risk Register

Documento vivo que registra riesgos, probabilidades, impactos (EMV), responsables y planes de mitigación.

Release / Lanzamiento

Acción de poner en producción un conjunto de funcionalidades verificadas; incluye planificación, pilotaje, despliegue y post-lanzamiento.

Release Notes (Notas de lanzamiento)

Documento que resume funcionalidades nuevas, correcciones, impactos y pasos requeridos por usuarios tras un lanzamiento.

Retrospective (Retrospectiva)

Evento donde el equipo inspecciona su proceso y acuerda acciones de mejora. Debe producir acciones con dueños y criterios de éxito.

Riesgo (Risk)

Evento o condición incierta que, si ocurre, afecta negativamente (o positivamente) los objetivos; gestionado con identificación, análisis, respuesta y monitoreo.

Roadmap

Plan estratégico que muestra la evolución del producto a mediano/largo plazo con hitos y releases proyectados.

ROI (Return on Investment)

Indicador financiero que relaciona beneficio neto con inversión; usado en justificaciones de negocio.

Rollback

Procedimiento para revertir un despliegue cuando indicadores críticos exceden umbrales; debe estar probado y documentado.

Runbook / Playbook

Documento operativo que contiene pasos para manejar eventos frecuentes o incidentes (incluye comandos, contactos y pruebas).

Scrum

Marco ágil empírico que define roles (PO, SM, dev team), eventos (sprint planning, daily, review, retro) y artefactos (backlogs, increment) para entrega iterativa.

Scrum Master (SM)

Rol responsable de asegurar que Scrum funcione: facilita eventos, elimina impedimentos y protege al equipo de distracciones.

Scope creep

Tendencia a incorporar cambios no controlados que expanden el alcance del proyecto sin la debida priorización ni recursos.

Sensibilidad (Análisis de sensibilidad)

Técnica para evaluar cómo cambian resultados (p. ej., payback, ROI) ante variaciones en supuestos clave.

Spike

Actividad temporal de investigación o prototipado para reducir incertidumbre y obtener una estimación más fiable.

Sprint

Iteración fija (1–4 semanas) en la que el equipo entrega un incremento potencialmente entregable; unidad básica de cadencia en Scrum.

Sprint Goal

Objetivo consensuado para el sprint que orienta la selección de historias y la toma de decisiones durante la ejecución.

Sprint Planning

Evento inicial del sprint para decidir el alcance (qué se hará) y el plan (cómo se hará), resultando en Sprint Backlog y Sprint Goal.

Stakeholder (Parte interesada)

Persona u organización con interés o influencia en el proyecto (clientes, usuarios, sponsor, reguladores).

Story points

Unidad abstracta para estimar complejidad relativa, esfuerzo y riesgo de una historia; no corresponde directamente a horas.

Sustainability (Sostenibilidad del producto)

Capacidad del producto de mantenerse operando y siendo evolucionado en el tiempo sin depender de esfuerzos extraordinarios.

TDD (Test-Driven Development)

Práctica de escribir tests antes del código, lo que guía el diseño y mejora la cobertura y calidad.

Tradicional (Waterfall)

Modelo secuencial de desarrollo en fases (análisis, diseño, implementación, pruebas); contrasta con enfoques iterativos/ágiles.

Trunk-based development

Estrategia de branching que favorece integraciones frecuentes en una rama principal (trunk) para reducir costes de integración.

User Story (Historia de usuario)

Descripción corta de una funcionalidad vista desde el usuario: “Como [rol], quiero [acción] para [beneficio]”, acompañada de criterios de aceptación.

Velocity (Velocidad)

Media de puntos completados por sprint; usada para pronosticar capacidad y duración de releases (debe usarse con prudencia).

WIP (Work In Progress)

Trabajo en curso; limitar WIP ayuda a mejorar flujo y reducir multitarea.